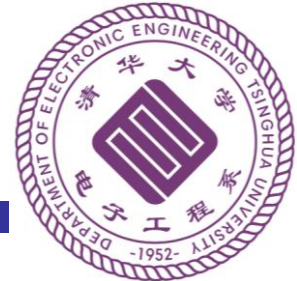




数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第七讲 流水线技术

第3部分 流水线异常控制、高级处理器技术

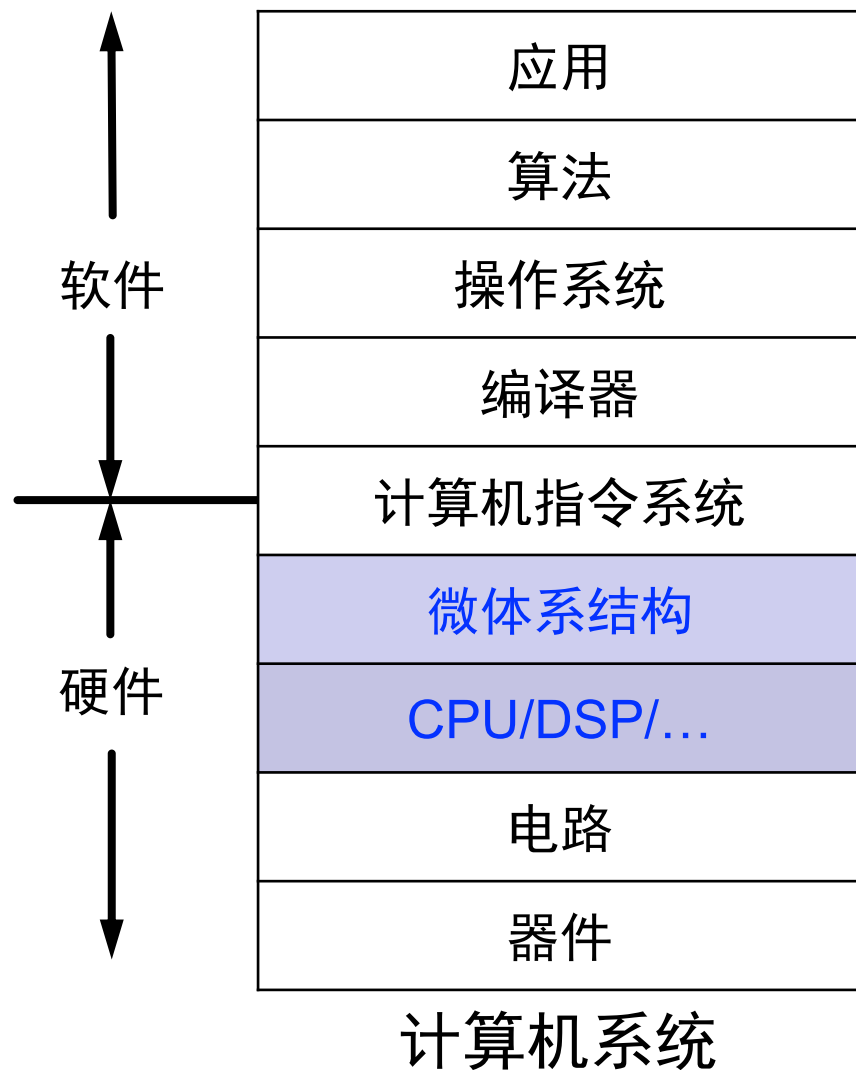
李学清

#腾讯会议： 453-1488-6548

助教：唐文骏*/陈仲豪/曾令安

2024年12月9日

微体系结构

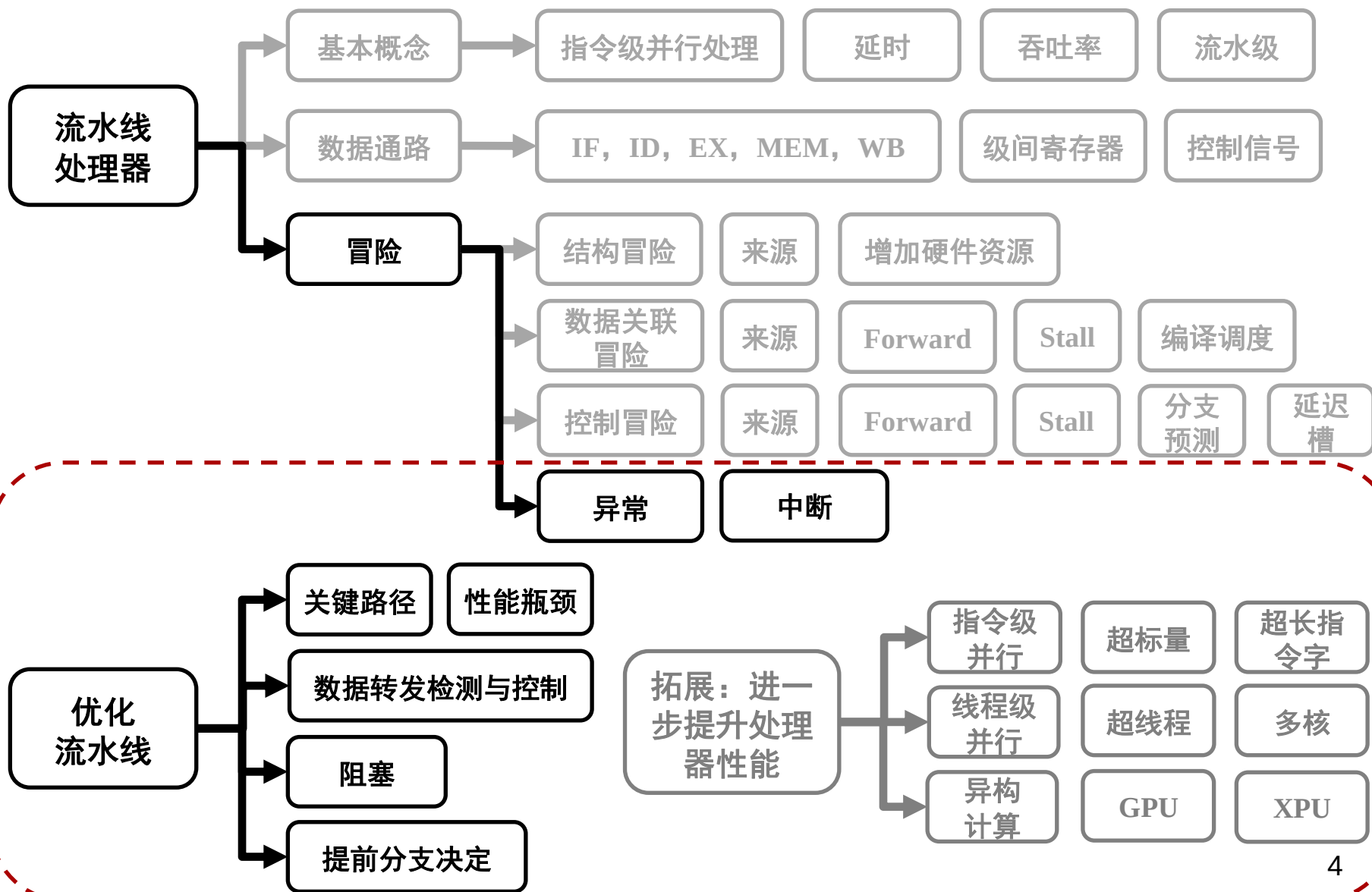


周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	11/3	期中考试
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/9	流水线/存储器
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

本节课学习目标

- **知识点：概念、原理和指标**
 - 单周期、多周期、流水线处理器的区别
 - 流水线的基本概念、数据通路
 - 流水线中的冒险起因与分类
- **能力**
 - 掌握设计和分析流水线处理器的方法
 - 掌握分析和解决流水线中冒险的方法
- **思想**
 - 时空复用
 - 并行加速

知识点框图



回顾：数据冒险解决方法

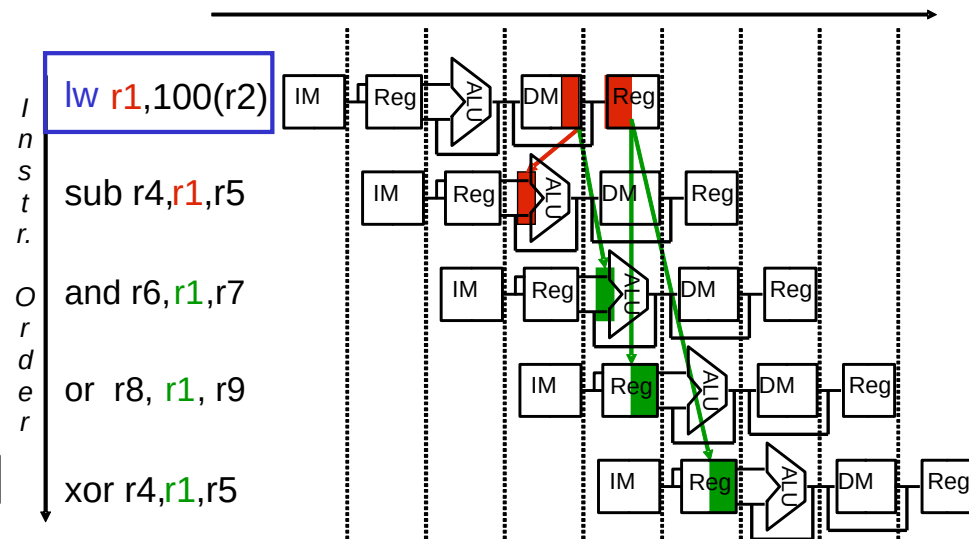
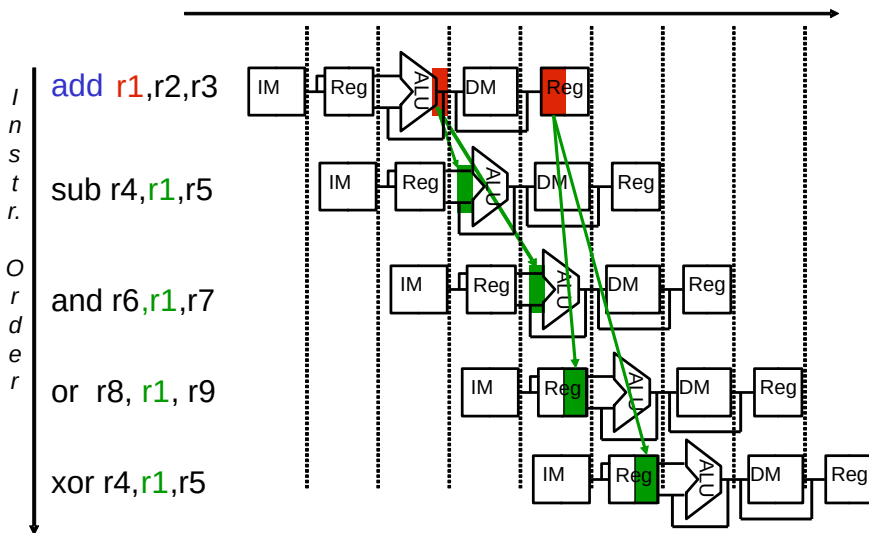
• 数据转发

Stage	R-type	Load	Store	Branch
IF	IR <= MemInst[PC]; PC <= PC+4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]]; Branch: If(A == B) PC <= PC + signext(IR[15:0]) << 2 Jump: If(JUMP) PC<=NewPC EX/MEM.ALUOut			
EX	ALUOut <= A op B EX/MEM.ALUOut MEM/WB.ALUOut MDR	ALUOut <= A + signext(IR[15:0]) EX/MEM.ALUOut MEM/WB.ALUOut MDR		
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B MDR	
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		

讨论：何时转发？转发哪个？是否完备？

回顾：数据转发Datapath

- 数据关联时提前提供数据
 - EX阶段结束前，ALU 运算结果已经就绪
 - MEM阶段结束前，LW指令读取的数据已经就绪



回顾：数据转发检测和控制

1. EX/MEM hazard:

```
if (EX/MEM.RegWrite
and (EX/MEM.RegWrAddr != 0)
and (EX/MEM.RegWrAddr == ID/EX.RegisterRs))
    ForwardA = 10
if (EX/MEM.RegWrite
and (EX/MEM.RegWrAddr != 0)
and (EX/MEM.RegWrAddr == ID/EX.RegisterRt))
    ForwardB = 10
```

将结果从前一条更新寄存器（且不是\$0）的指令回馈到ALU输入端

2. MEM/WB hazard:

```
if (MEM/WB.RegWrite
and (MEM/WB.RegWrAddr != 0)
and (MEM/WB.RegWrAddr == ID/EX.RegisterRs)
and (EX/MEM.RegWrAddr != ID/EX.RegisterRs || ~ EX/MEM.RegWrite))
    ForwardA = 01
```

```
if (MEM/WB.RegWrite
and (MEM/WB.RegWrAddr != 0)
and (MEM/WB.RegWrAddr == ID/EX.RegisterRt)
and (EX/MEM.RegWrAddr != ID/EX.RegisterRt || ~ EX/MEM.RegWrite))
    ForwardB = 01
```

将结果从前前条更新寄存器（且不是\$0）的指令回馈到ALU输入端

回顾：控制冒险解决方法

- 在ID阶段加入分支判断
- 提前分支判断

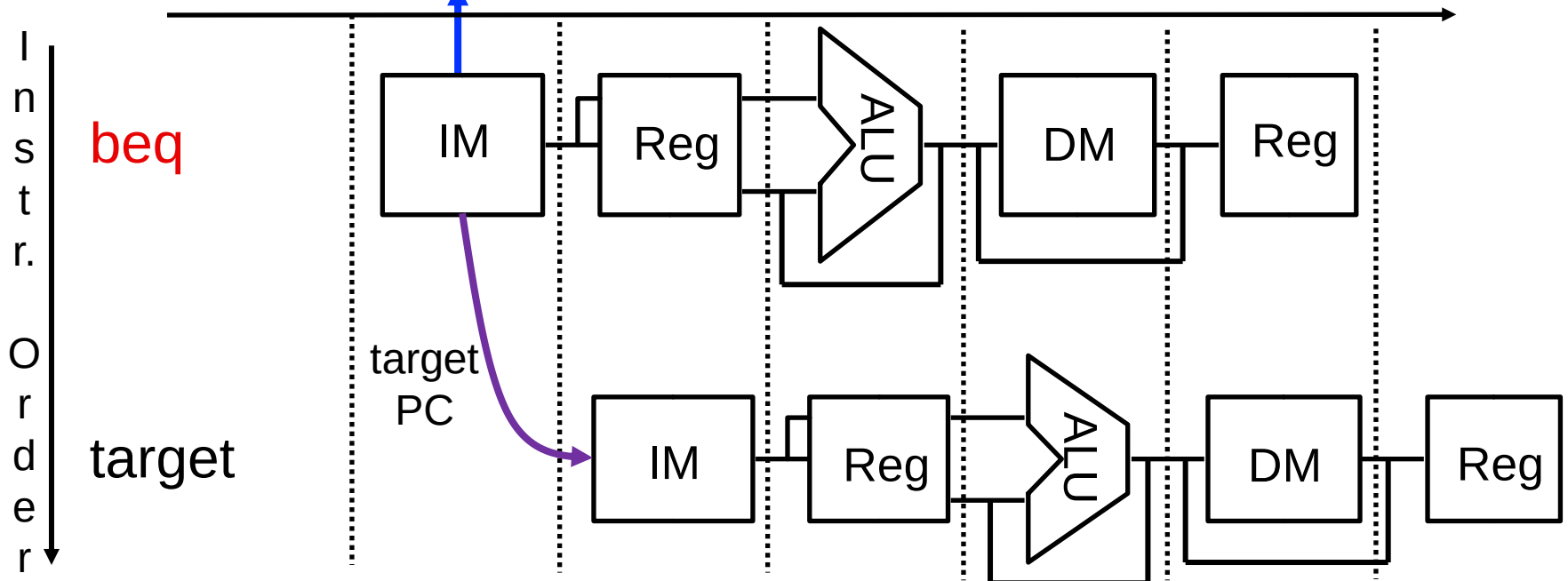
Stage	R-type	Load	Store	Branch
IF	IR <= MemInst[PC]; PC <= PC+4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]]; Branch: If(A == B) PC <= PC + signext(IR[15:0]) << 2 Jump: If(JUMP) PC<=NewPC EX/MEM.ALUOut			
EX	ALUOut <= A op B EX/MEM.ALUOut MEM/WB.ALUOut MDR	ALUOut <= A + signext(IR[15:0]) EX/MEM.ALUOut MEM/WB.ALUOut MDR		
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B MDR	
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		8

回顾：动态分支预测

• 动态分支预测流程：Clock 1

– beq指令的IF阶段

- 查询指令地址是否在BHT和BTB中
- 若存在，**根据历史记录判断是否跳转**：若跳转，取出目标地址作为下一条指令的IF地址；若不存在，预测PC为PC+4

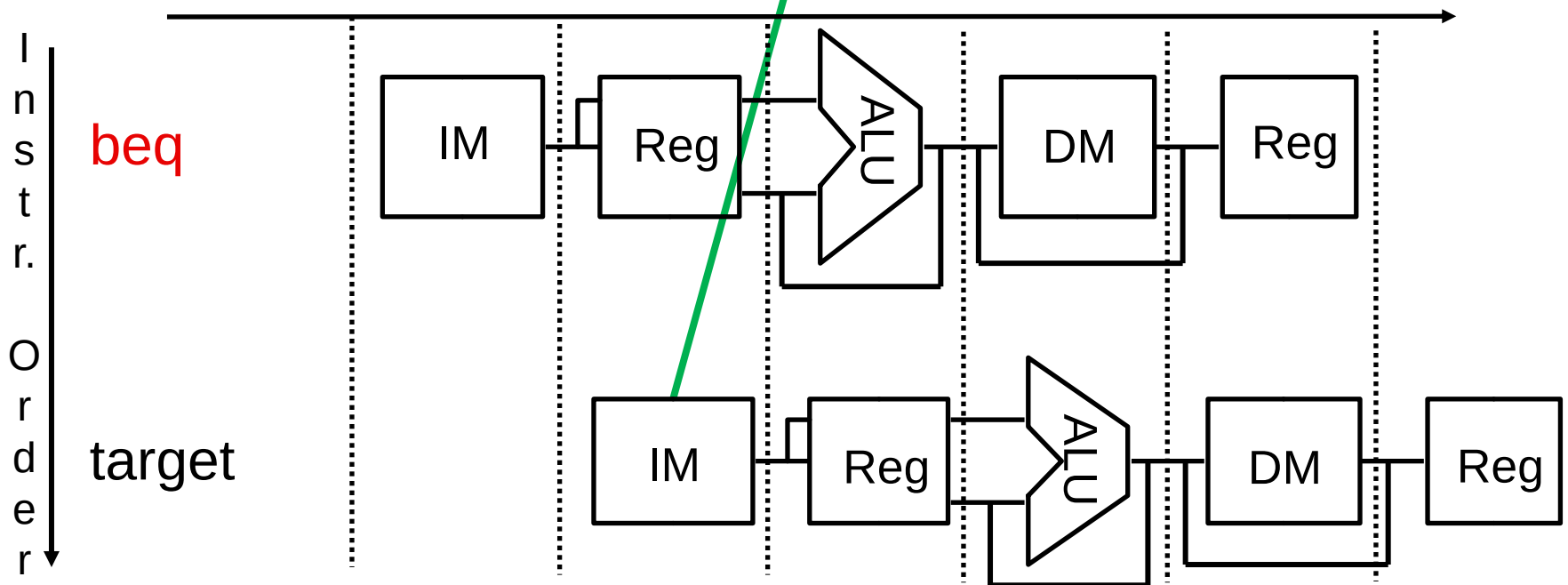


回顾：动态分支预测

- 动态分支预测流程：Clock 2

- beq指令的ID阶段

- 下一条指令的IF段根据预测目标地址取出指令



本节课目录

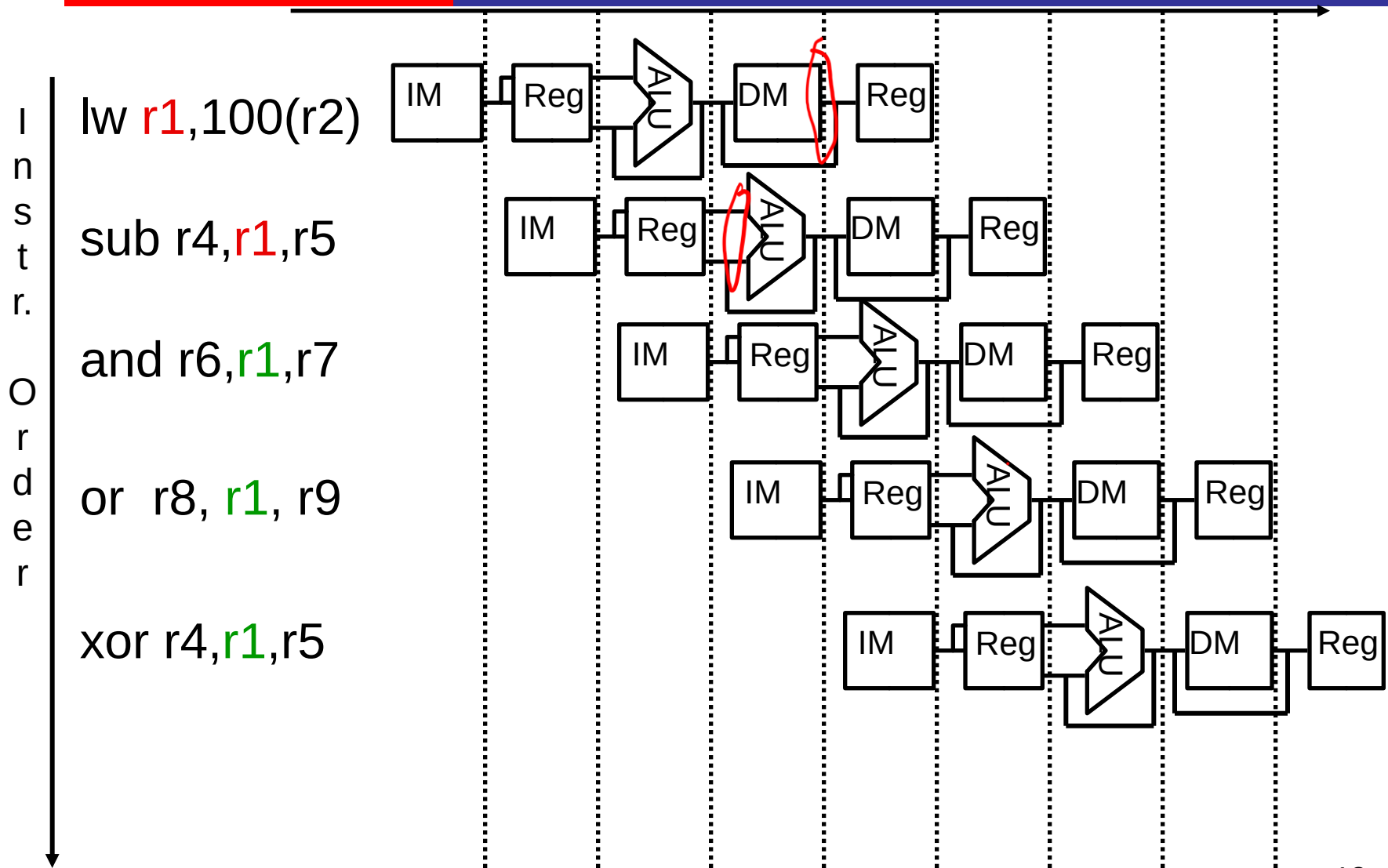
- 复习与回顾：流水线的冒险 P5
- 数据通路的实现（考虑冒险）
 - 结构冒险
 - 数据冒险 P12
 - 控制冒险 P28
- 进一步提升处理器性能 P60

流水线数据通路的实现（考虑冒险）

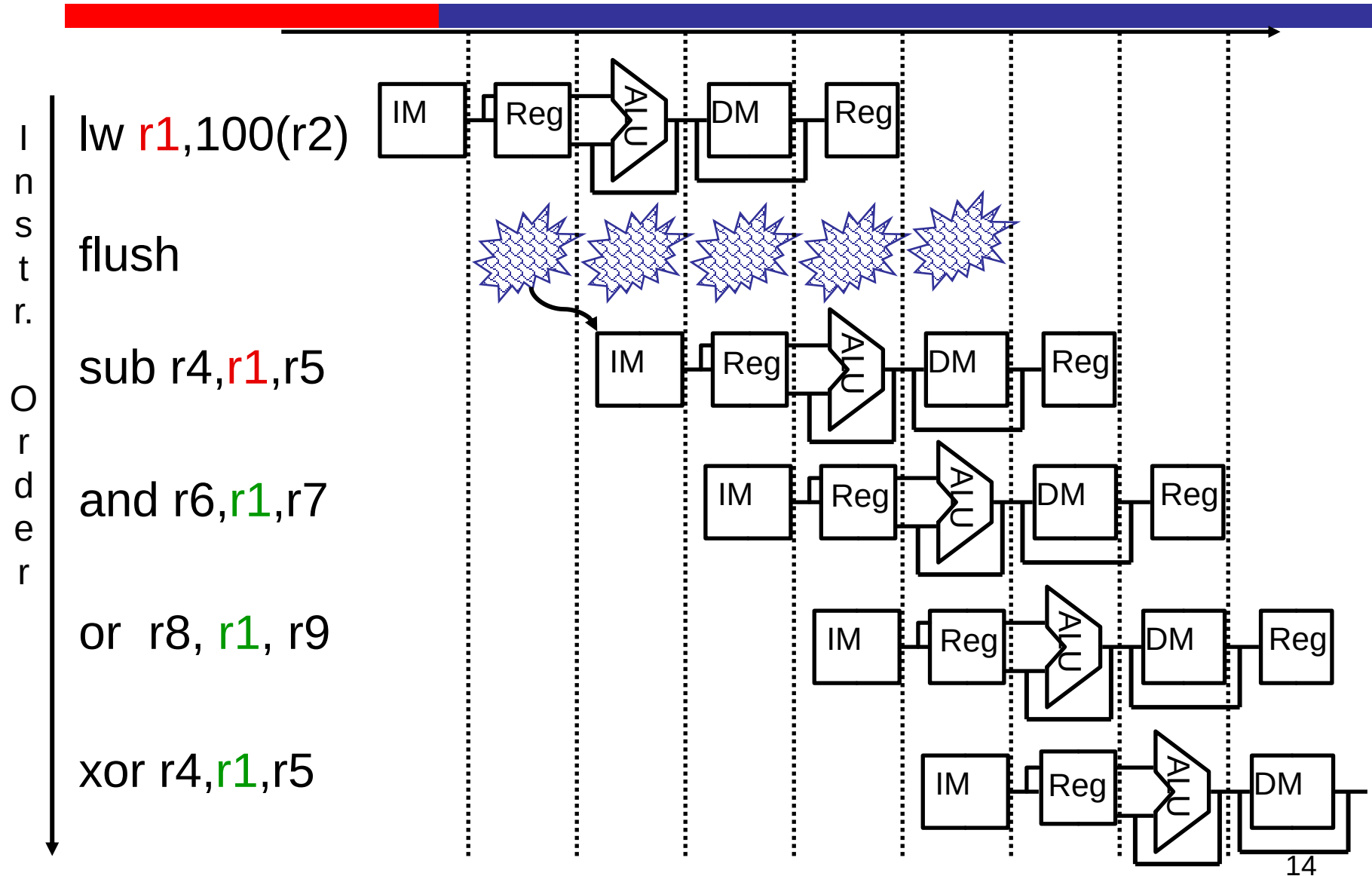
关键：数据通路和控制信号的设计

- 控制信号如何随流水线运行？
 - 在各个流水级之间增加控制信号的寄存器
 - 对应级用对应控制信号
 - 不再使用的控制信号可不再传递
- 将各个资源和状态(流水级)联系起来
 - 从设计上要避免结构冒险的出现：保证资源是够用的
- 找到所有的数据冒险和控制冒险问题
 - 如果是寄存器关联引起的 → data hazard: forward/stall
 - 如果是和PC有关的竞争 → control hazard

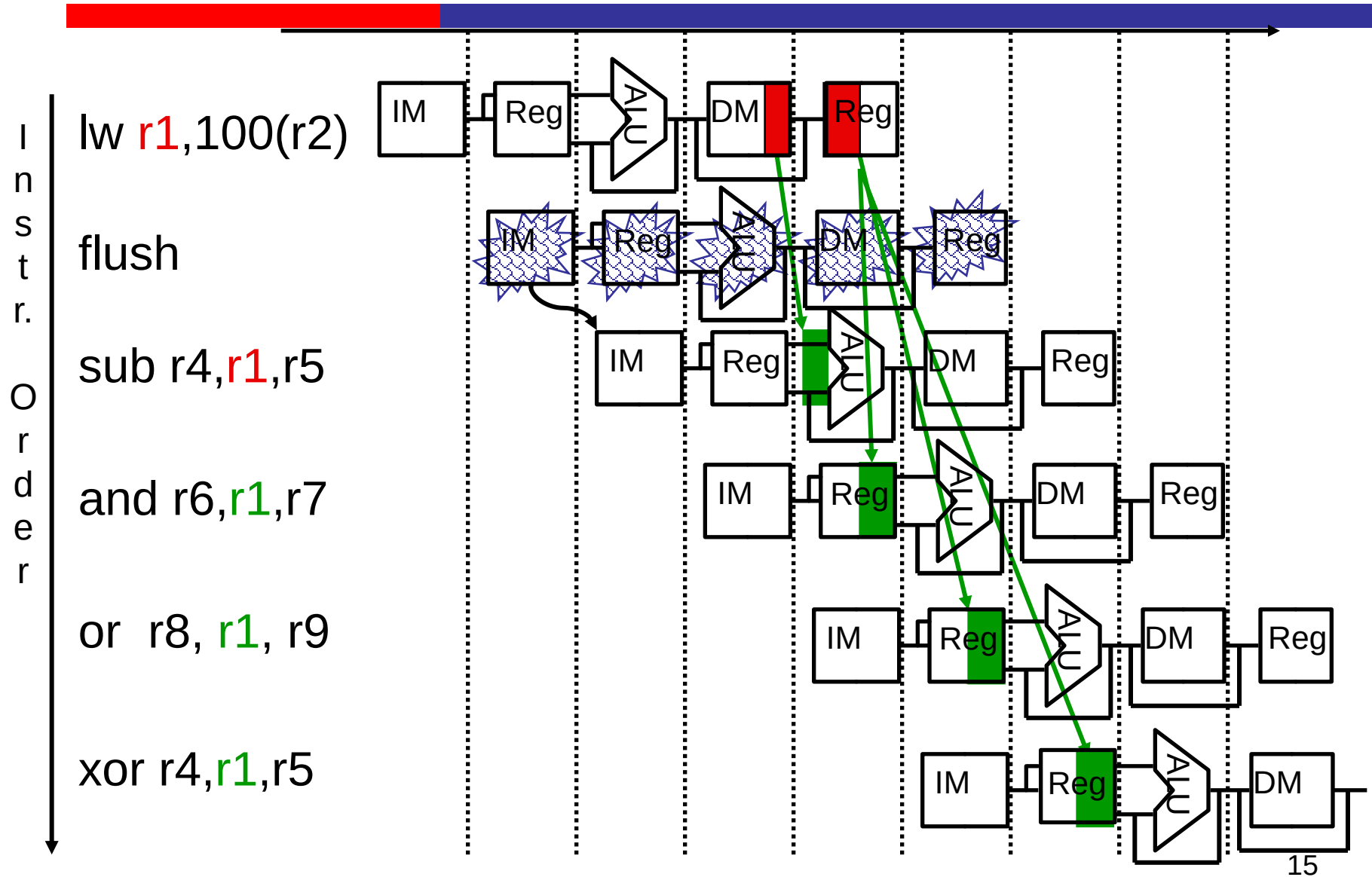
数据转发: Load-use Hazards



数据转发: Load-use Hazards



数据转发: Load-use Hazards

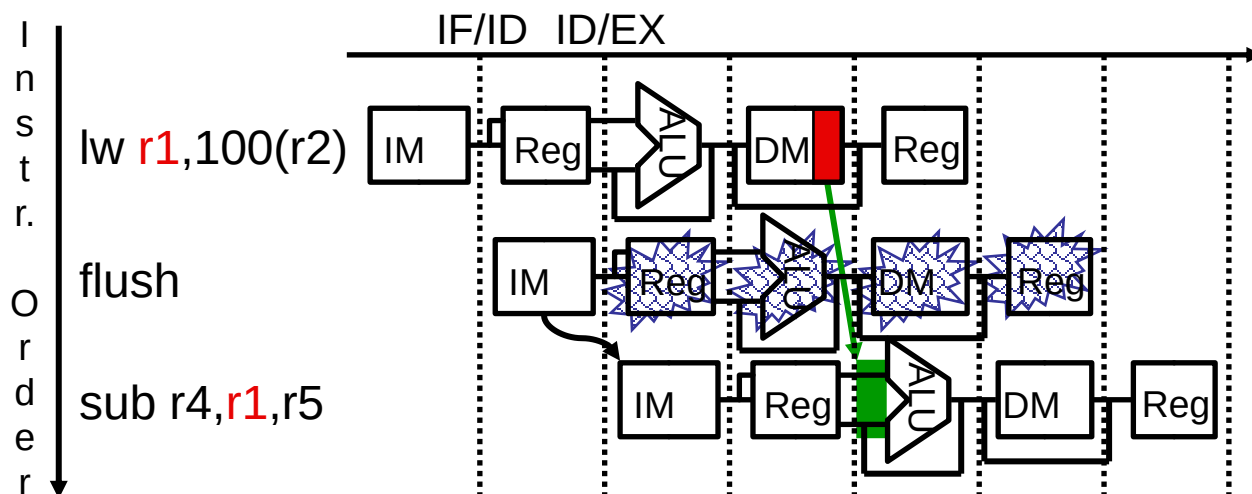


Load-use Hazard 检测

- 因为load-use hazard无法通过forward完全解决, 即便加了forward电路仍然存在hazard, 必须加一个阻塞周期
 - 先检查是否是load指令: if (ID/EX.MemRead)
 - 再检查EX级load指令的rt是否与ID阶段两个rs/rt reg index一样
 - 经检测导致的阻塞, 之后forward逻辑就可以处理剩下的冒险问题了

Detailed ID Hazard Detection:

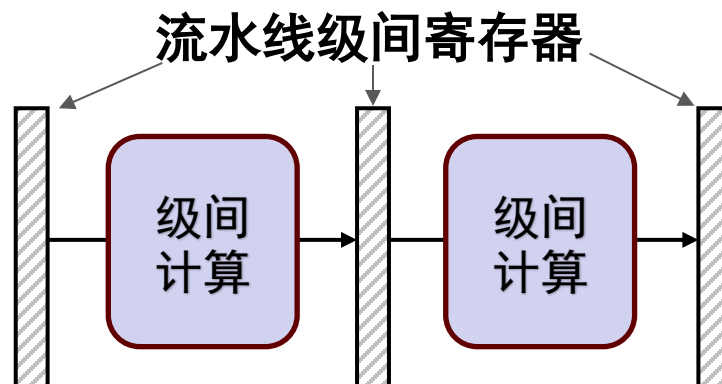
if (ID/EX.MemRead and ((IF/ID.RegisterRs == ID/EX.RegisterRt) or (IF/ID.RegisterRt == ID/EX.RegisterRt))) Stall the pipeline



Stall the pipeline的
具体操作是什么？

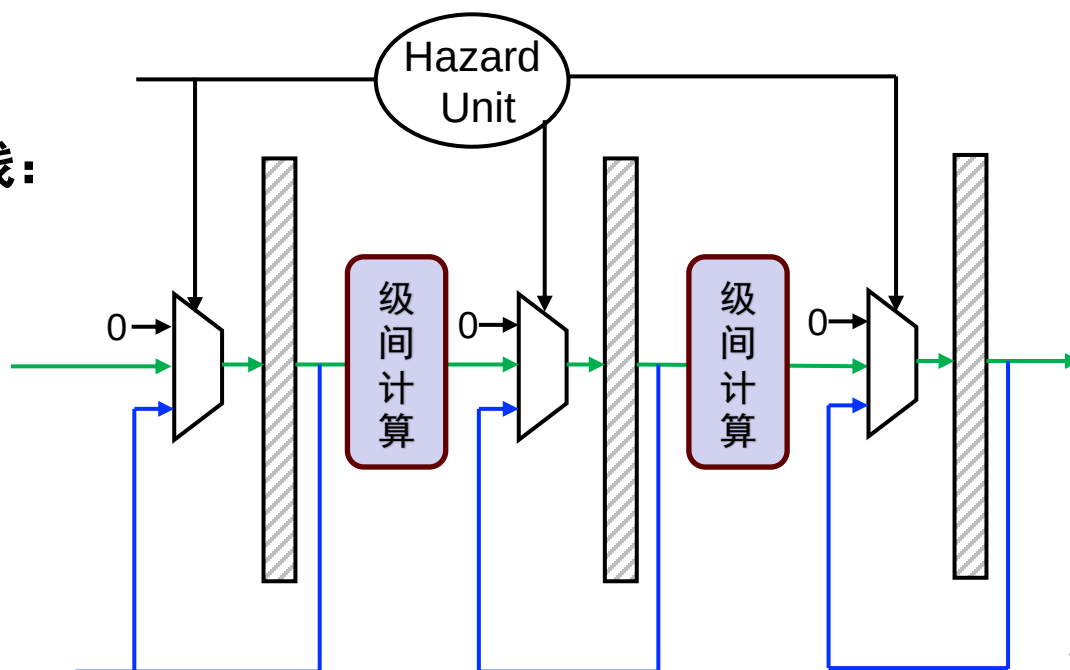
Load-use Hazards 阻塞

• 阻塞、清空 of 硬件实现



加入阻塞、清空后的流水线：
级间寄存器添加Mux

- 来自上一级计算结果
(不阻塞/清空)
- 来自本级输出
(阻塞)
- 来自0输入
(清空)

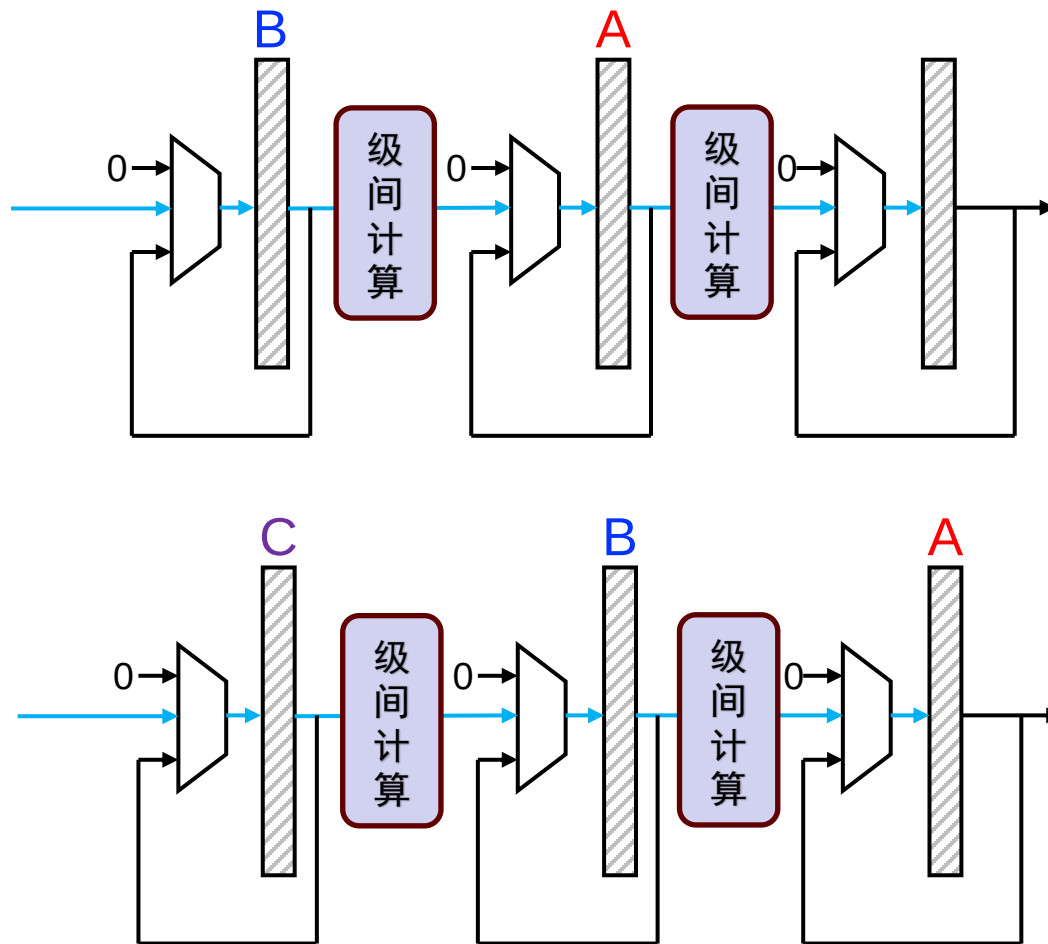


Load-use Hazards 阻塞

- 阻塞、清空 of 硬件实现

- 正常执行的流水线

– A – B – C



Load-use Hazards 阻塞

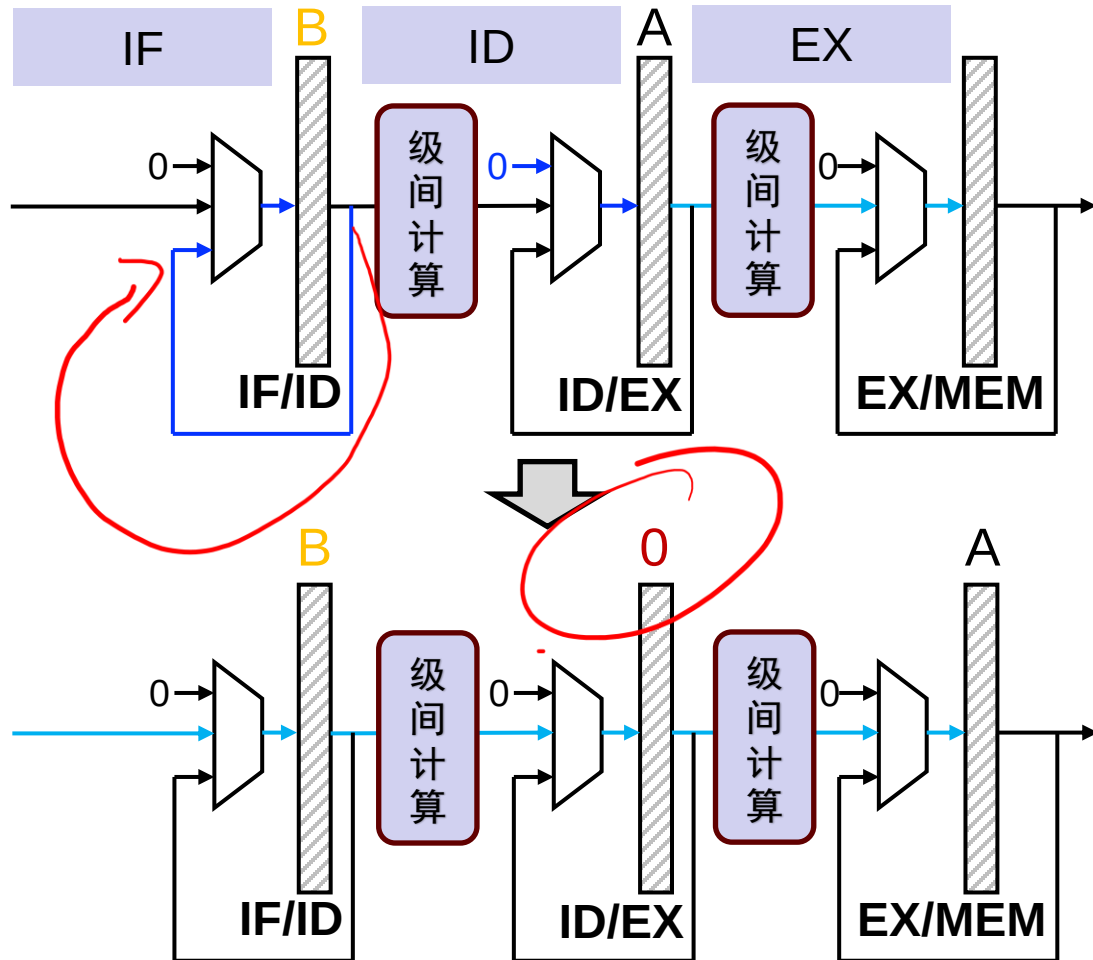
• 阻塞、清空硬件实现

当A是load，B是use
B需要stall，具体方法：

- A与B之间插入nop：
flush ID/EX
- 暂停B及之后的指令：
keep IF/ID、为IF阶段
保持PC

执行顺序：

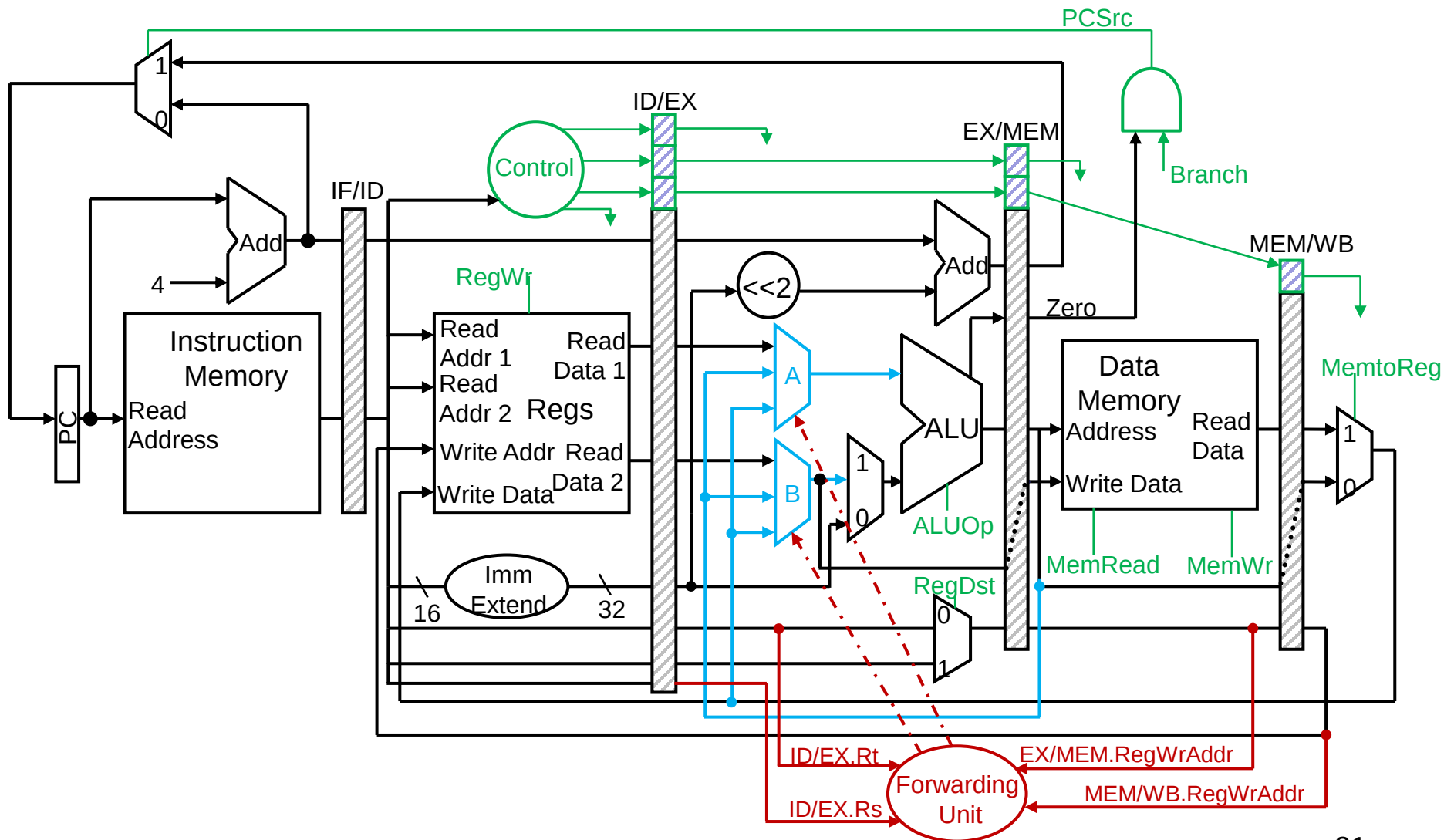
A – 0/nop – B



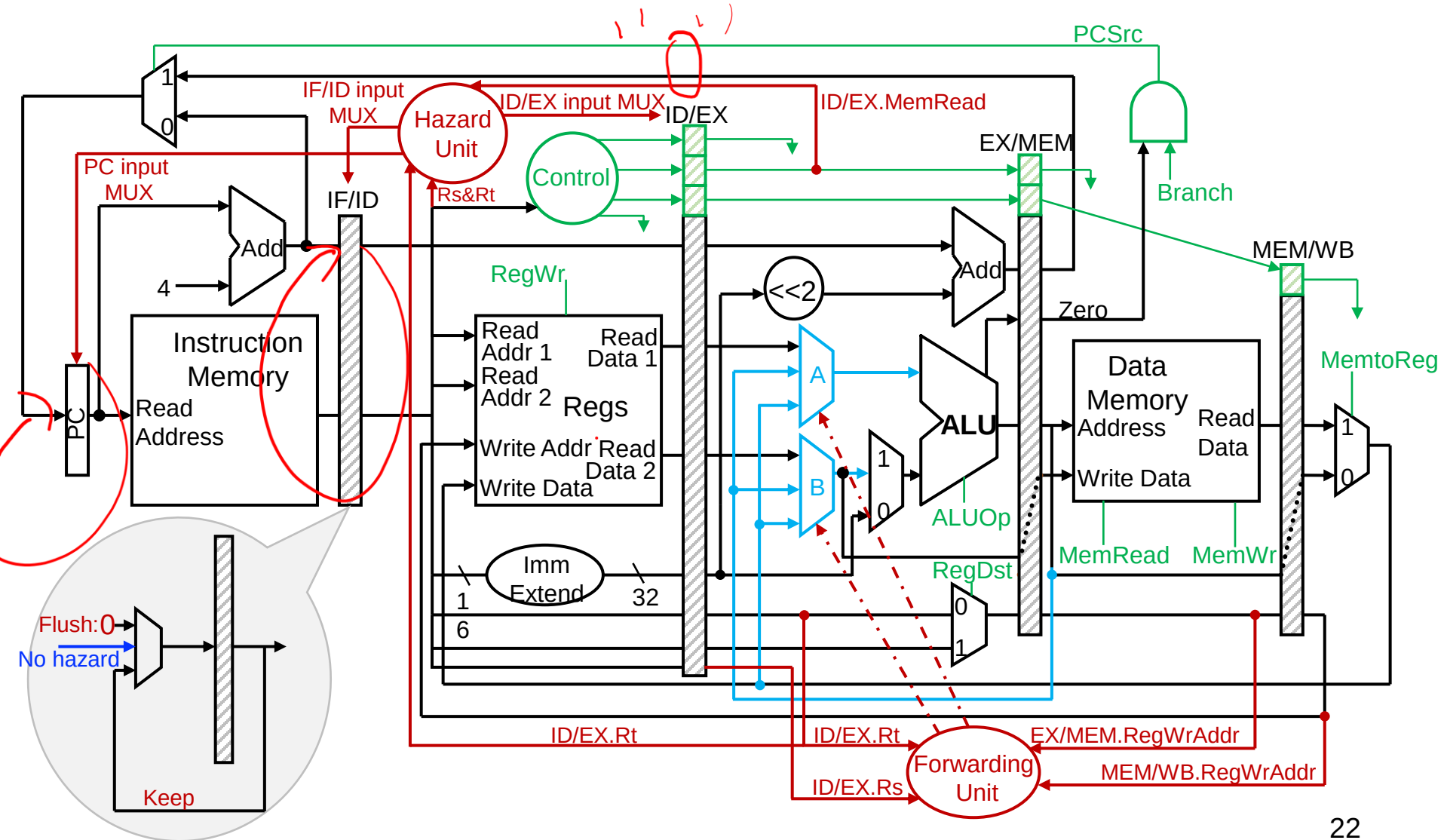
Load-use Hazards 阻塞

- 必须实现阻塞功能才能使forward发挥作用
- 阻止IF/ID 阶段的指令进入EXE阶段执行,
 - 只要保持PC寄存器和IF/ID流水线状态寄存器不变
 - 具体地, 由前面的hazard检测电路来控制这些寄存器的写入MUX
- 从EX开始的后半段在阻塞时应处于清空状态: 执行NOP
 - 将 ID/EX pipeline register中与EX, MEM, WB有关的控制信号改为不工作状态 (不影响电路状态)
 - 假定控制信号为0总是使电路状态不改变:
 - 由Hazard检测电路控制选择使用正常的控制信号还是直接为0.

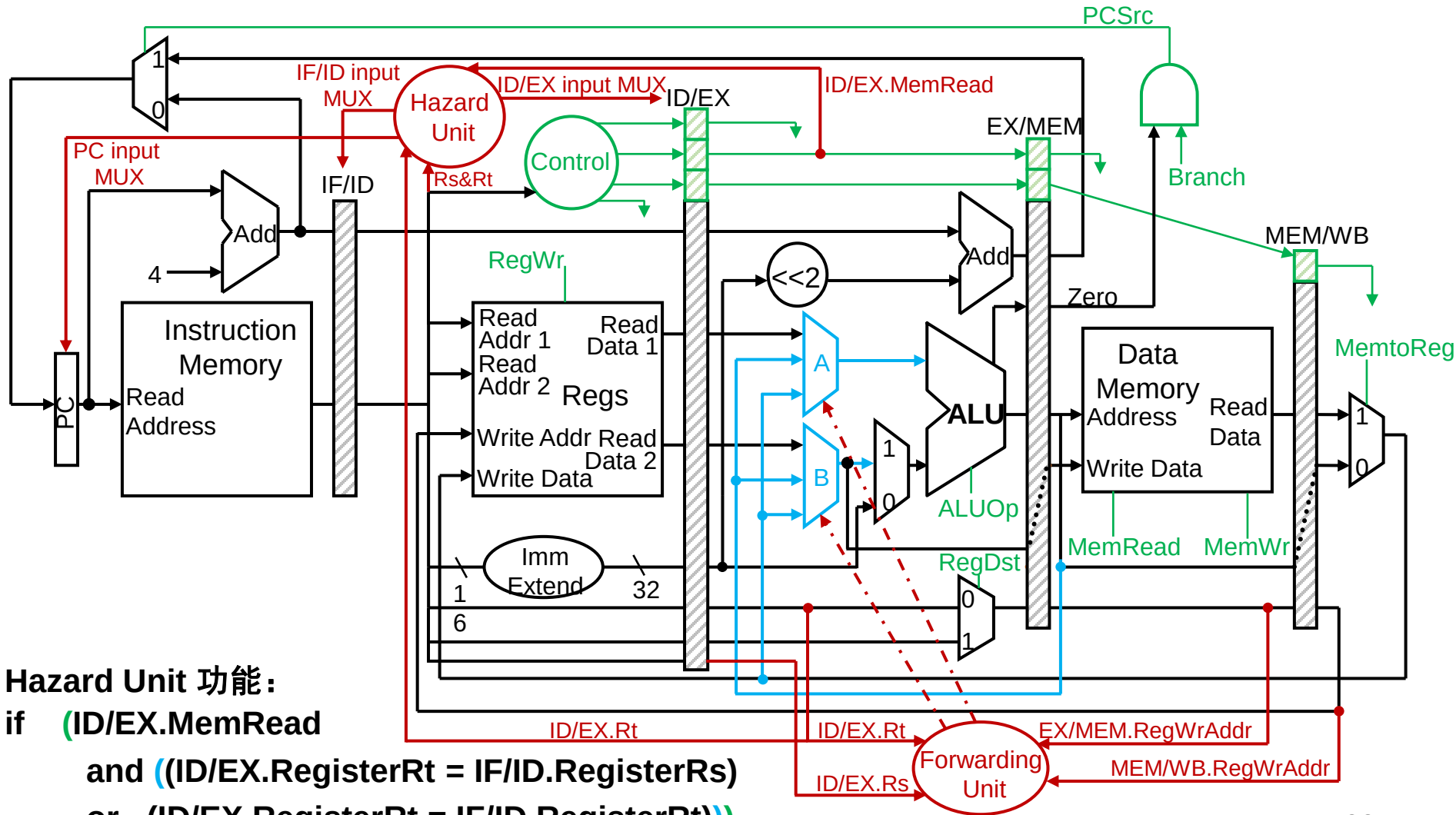
添加Hazard控制单元



添加Hazard控制单元



添加Hazard控制单元

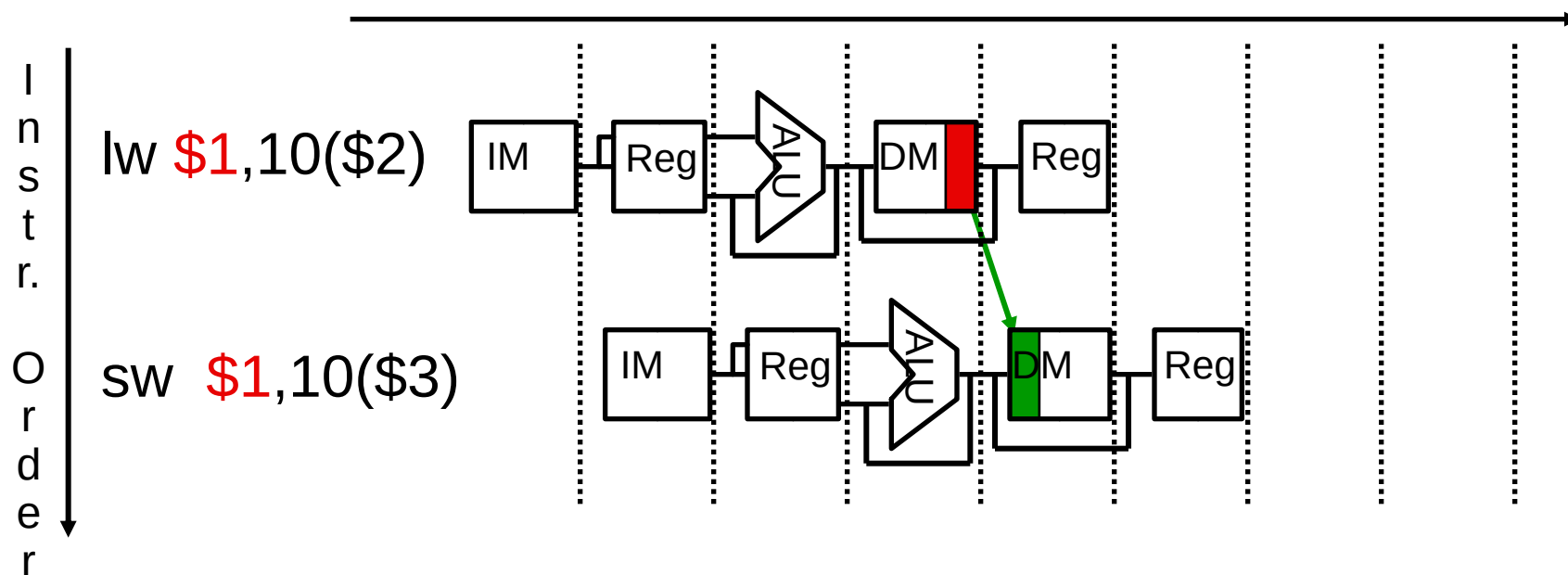


Hazard Unit 功能:
 if (ID/EX.MemRead
 and ((ID/EX.RegisterRt = IF/ID.RegisterRs)
 or (ID/EX.RegisterRt = IF/ID.RegisterRt)))

Keep IF/ID; Keep PC; Flush ID/EX;

存储器到存储器复制

- 若load-use hazard是由存储器复制引起的（不含读出数用来计算新写入地址的情形），可不必阻塞这一个周期
 - 在MEM阶段增加forward，可减少一个阻塞周期
 - 但是，如果下面实例中的\$1被用于计算sw的地址，其仍然是load-use hazard



考虑如下的汇编程序代码段，请问如何调整流水线执行指令的顺序，可以利用前述技术，保证执行正确且执行时间尽量短？

A 3-2-1-4-5

B 1-3-4-2-5

C 1-5-3-2-4

D 3-1-4-2-5

① lw \$2, 100(\$6)

② add \$4, \$2, \$5

③ lw \$3, 200(\$7)

④ add \$6, \$3, \$5

⑤ sub \$8, \$4, \$6

答案解析

- 考虑如下的汇编程序代码段，请问如何调整流水线执行指令的顺序，可以利用前述技术，保证执行正确且执行时间尽量短？

A. 3-2-1-4-5 (② before ①)

B. 1-3-4-2-5 (1-cycle stall between ③ and ④)

C. 1-5-3-2-4 (⑤ before ④)

D. 3-1-4-2-5 (no stall)

- A, C无法正确执行,

- B需要一个stall

① lw \$2, 100(\$6)

② add \$4, \$2, \$5

③ lw \$3, 200(\$7)

④ add \$6, \$3, \$5

⑤ sub \$8, \$4, \$6

本节课目录

- 复习与回顾：流水线的冒险 P5
- **数据通路的实现（考虑冒险）**
 - 结构冒险
 - 数据冒险 P12
 - **控制冒险 P28**
- 进一步提升处理器性能 P60

流水线的冒险

- **结构冒险 (Structural Hazard or Resource Conflict) : 资源矛盾!**
 - 两条指令在同一周期需要使用同样的硬件
 - 解决方法: 增加硬件资源; 调整指令占用资源的时间
- **数据关联冒险 (Data Hazard) : 操作数是否已经更新?**
 - 指令依赖于流水线中运行的其他指令结果
 - 解决方法: Stall; Forwarding; 编译器优化
- **控制冒险 (Control Hazard) : 下一条指令的PC是多少?**
 - 取指令依赖于流水线中指令的结果

流水线数据通路的实现（考虑冒险）

关键：数据通路和控制信号的设计

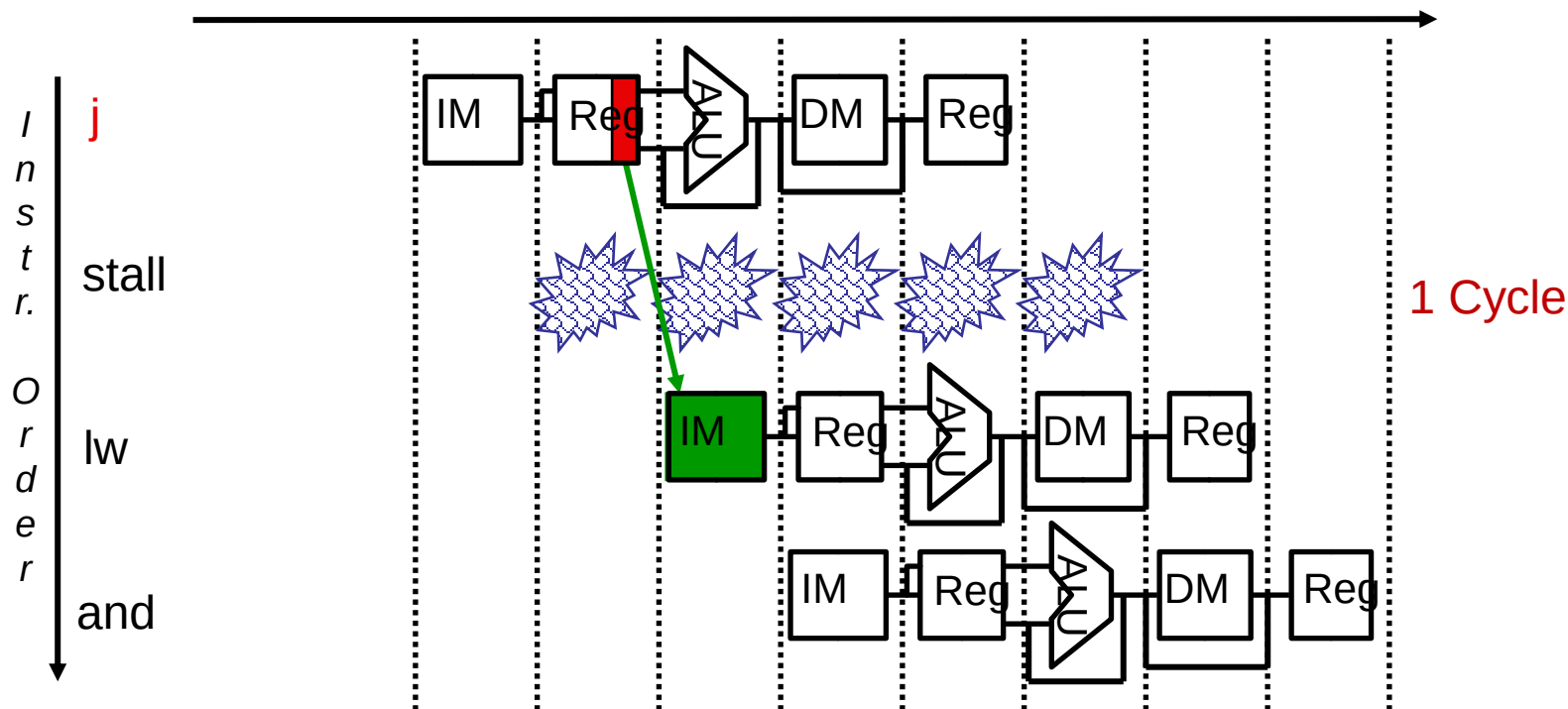
- 控制信号如何随流水线运行？
 - 在各个流水级之间增加控制信号的寄存器
 - 对应级用对应控制信号
 - 不再使用的控制信号可不再传递
- 将各个资源和状态(流水级)联系起来
 - 从设计上要避免结构冒险的出现：保证资源是够用的
- 找到所有的数据冒险和控制冒险问题
 - 如果是寄存器关联引起的=> data hazard: forward/stall
 - 如果是和PC有关的竞争=> control hazard

Control Hazards控制冒险

- 当指令流向不是预期的方式
 - 有条件的分支 (beq, bne)
 - 无条件的跳转 (j)
- 可能的解决方案
 - Stall
 - 将分支判断提前
 - 预测
 - 延迟决定, 即延迟分支(compiler support needed)
- 控制竞争不像数据竞争那样频繁的发生, 但是也没有像forwarding这样的有效的解决方式

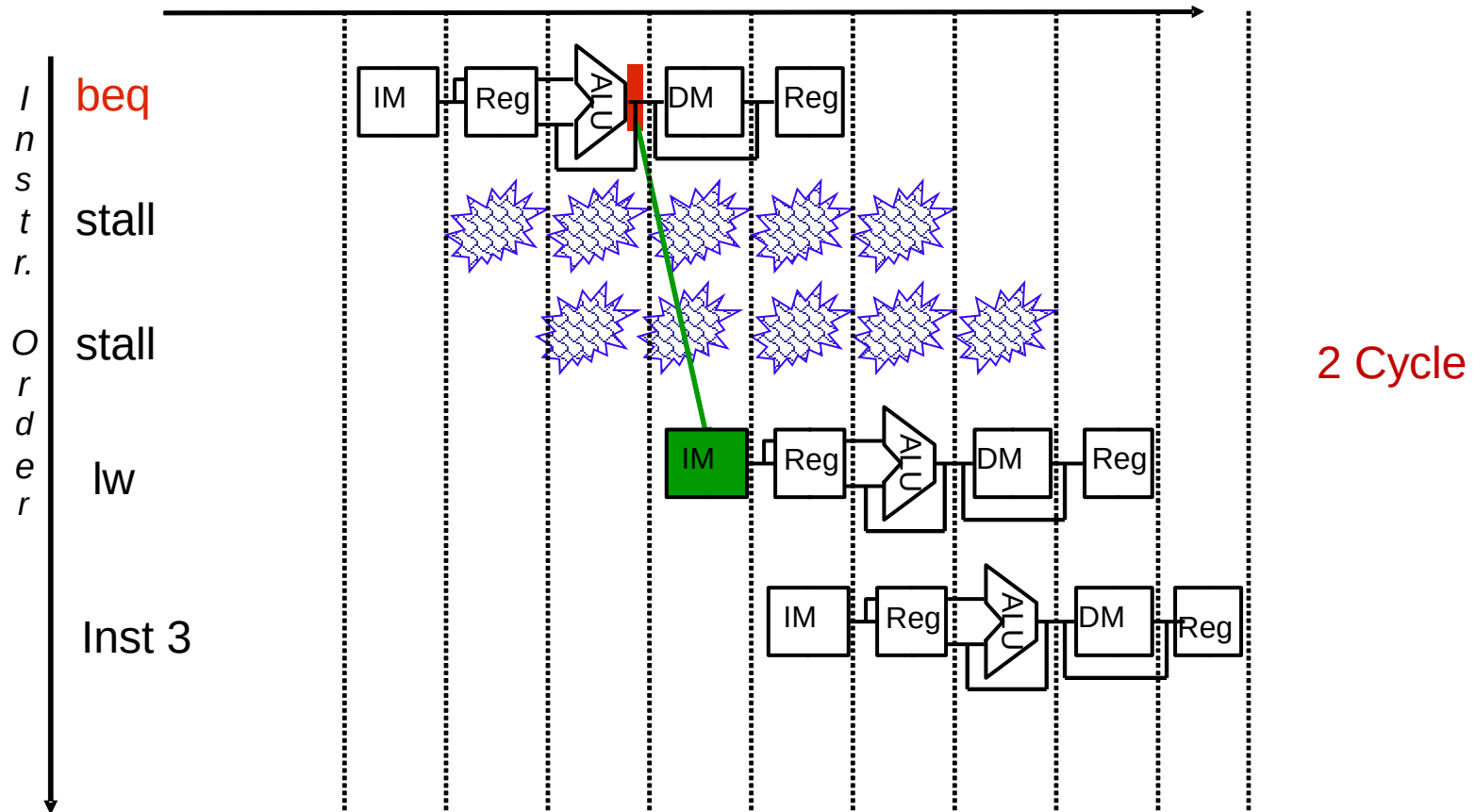
Jump 引起一个阻塞

- Jump 在ID后期可被判断出，只需阻塞一个周期
- 实际情况下, Jump比例不高（SPEC int instruction mix只有2%指令是Jump）



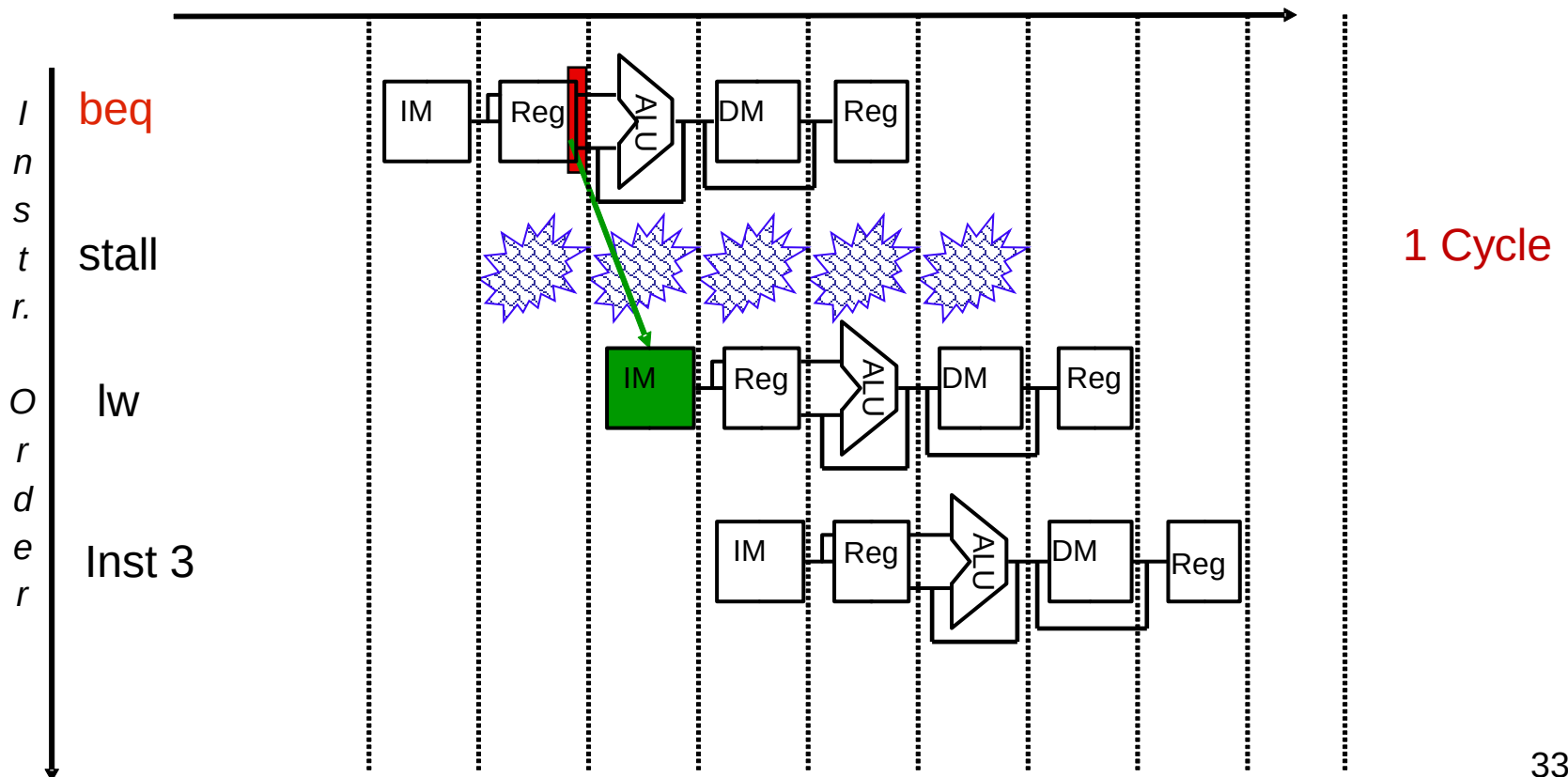
BEQ: Stall两个周期

- EX阶段判断分支结果, Stall减少为2个周期
- EX时序路径增加相应的硬件 (AND门, 2to1 MUX)

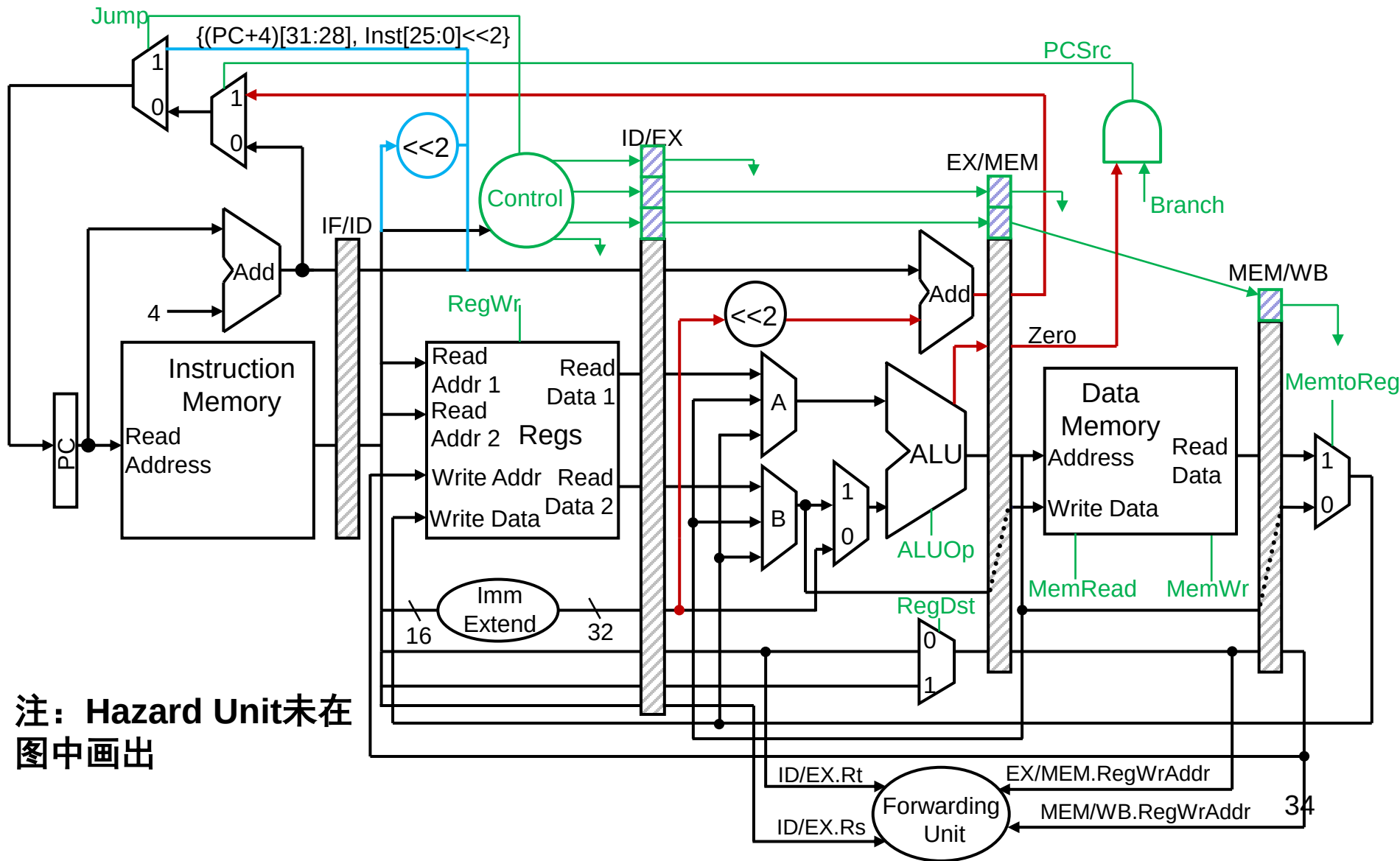


BEQ: Stall一个周期

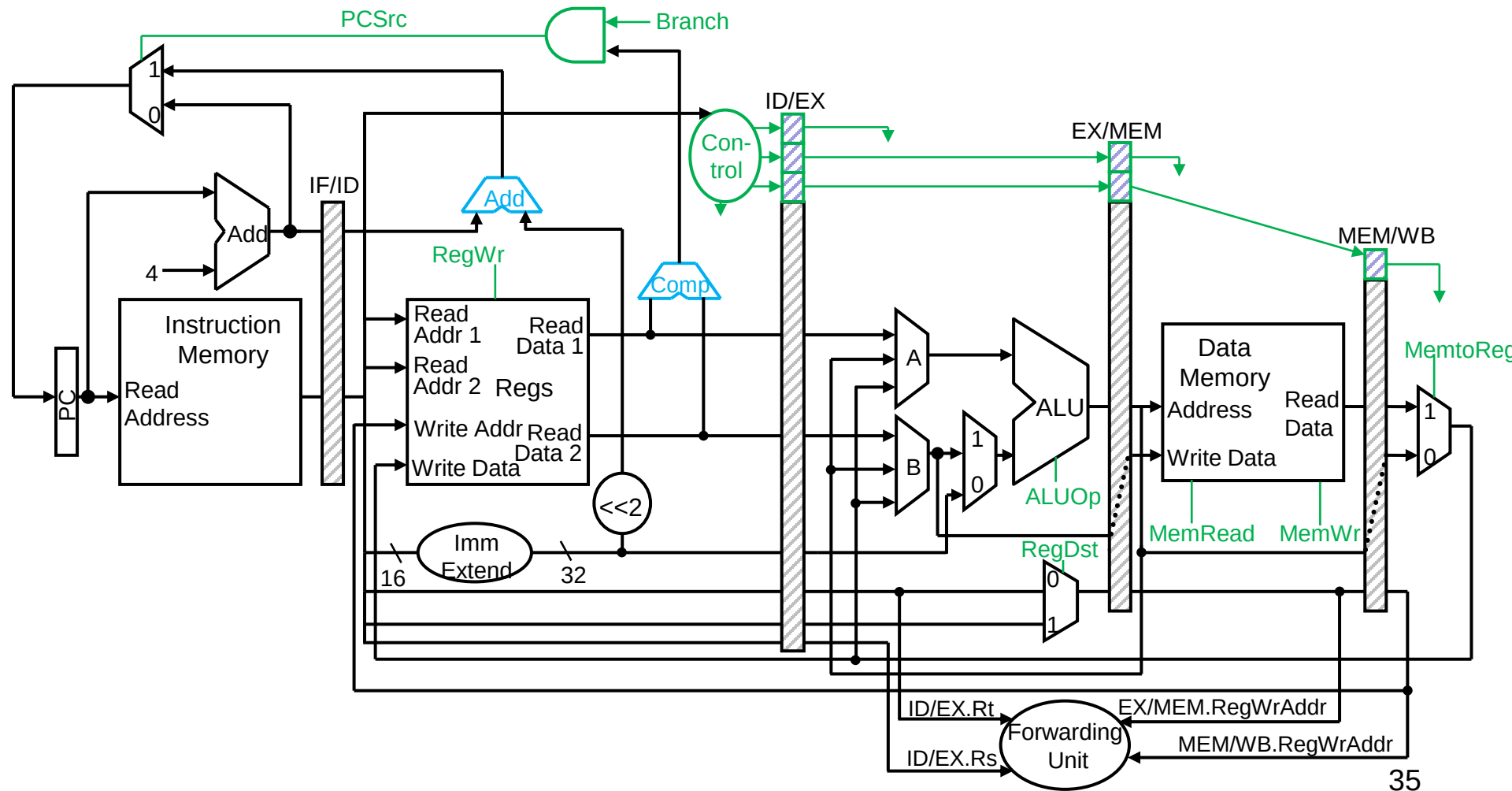
- **增加硬件：ID阶段就进行比较运算和有效地址运算**
 - 增加比较器，MUX，forwarding电路等硬件资源
 - Stall减少为1个周期



Datapath 分支与跳转电路

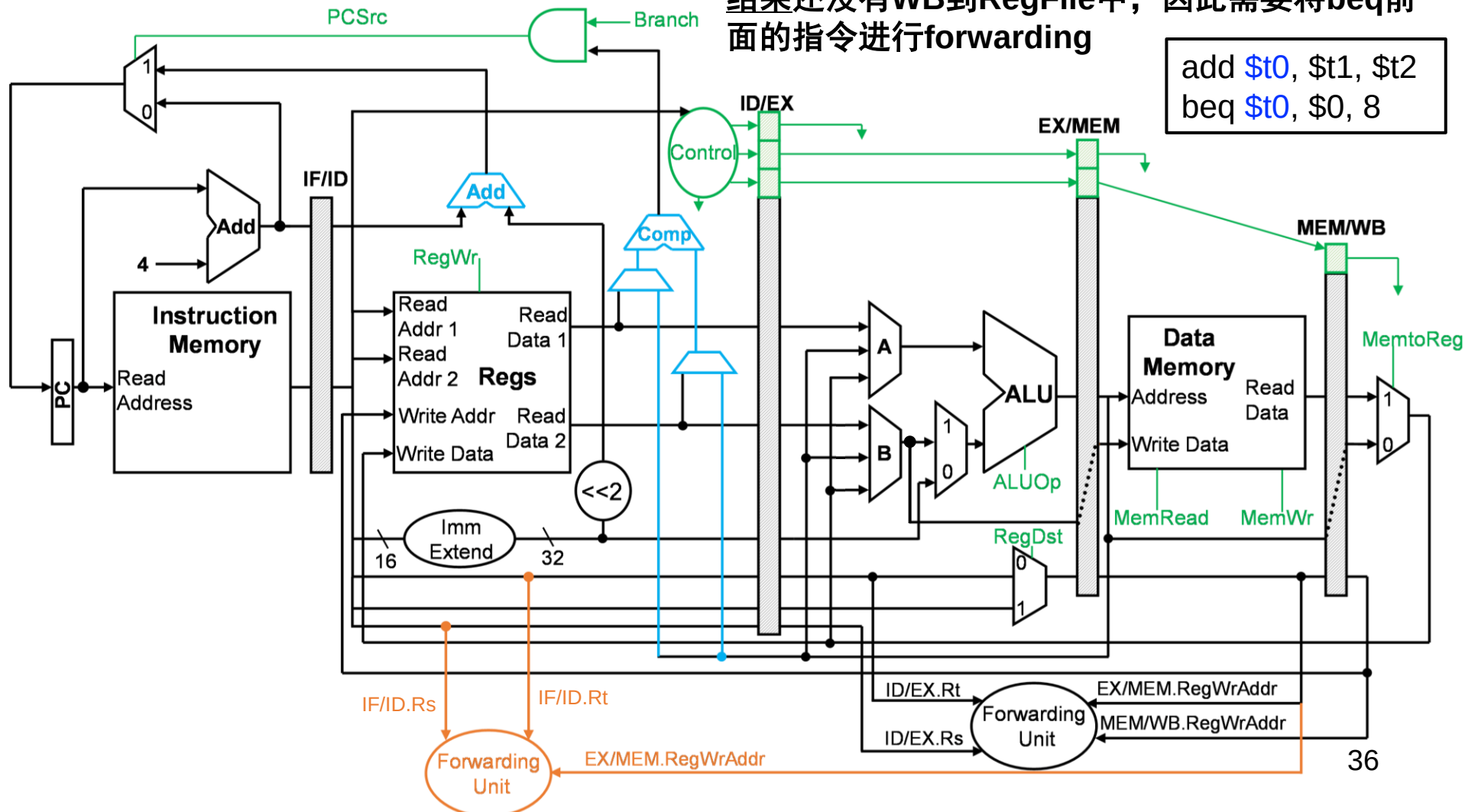


Control Hazard: ID stage 分支电路



Control Hazard: ID stage 分支电路

beq的判断提前, 使得beq可能依赖的前面指令的结果还没有WB到RegFile中, 因此需要将beq前面的指令进行forwarding



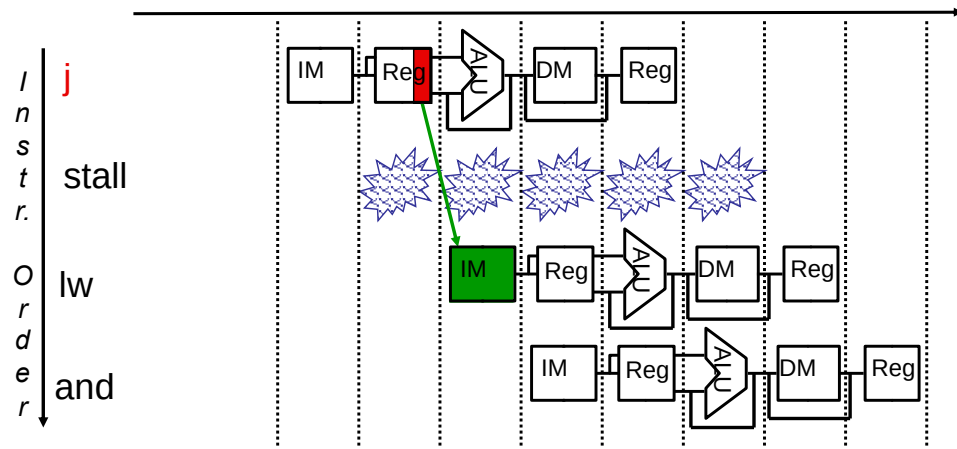
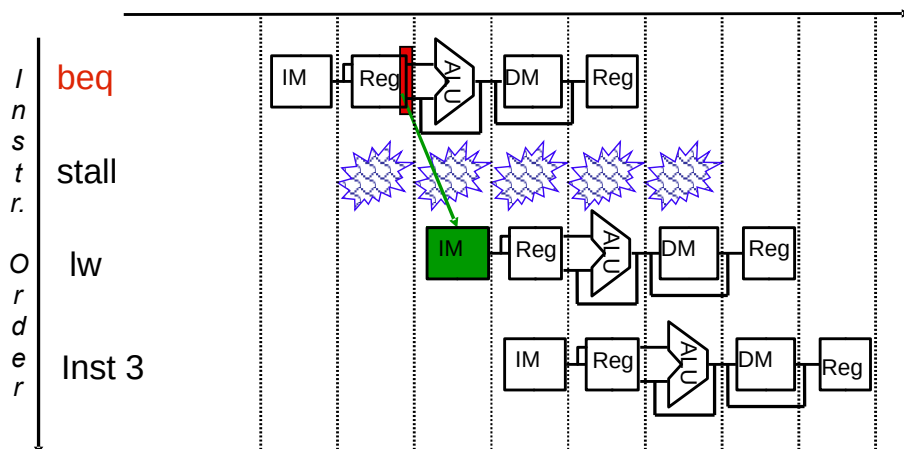
在流水线中提前进行分支决定

- 将分支判断放在EX阶段中
 - 可以将STALL减少为2个
- 增加硬件以在ID阶段就进行比较运算和有效地址的运算,
 - 这样可以使STALL减少为1
 - 与RF的读操作同时
 - 需要一个比较器，多路选择器等电路
 - 需要在ID阶段引入forwarding hardware电路
- 当流水线级数增加时,分支的判断更晚,需要更好的办法解决
 - 分支预测

控制冒险的解决方法

- **BEQ & J**

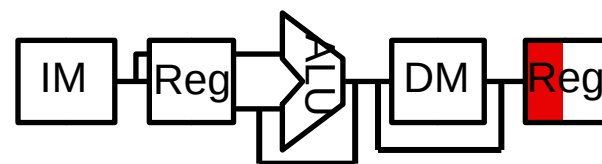
- BEQ在ID阶段就进行比较运算和有效地址运算
- Jump在ID后期可被判断出
- Stall减少为1个周期



如果仅考虑如下的汇编代码段，分支指令在ID阶段提前判断分支，以下说法**错误**的是：

```
①      lw r2,0(r1)
② label1: beq r2,r0,label2
③      add r3, r2, r1
④      beq r3,r0,label1
⑤      add r1,r3,r1
⑥ label2: sw r1,0(r2)
```

- ☐ A 需要增加EX/MEM到ID的转发
- ☐ B 不需要增加MEM/WB到ID的转发
- ☒ C 1与2之间需要stall 1个clock cycle
- ☐ D 3与4之间需要stall 1个clock cycle



答案解析

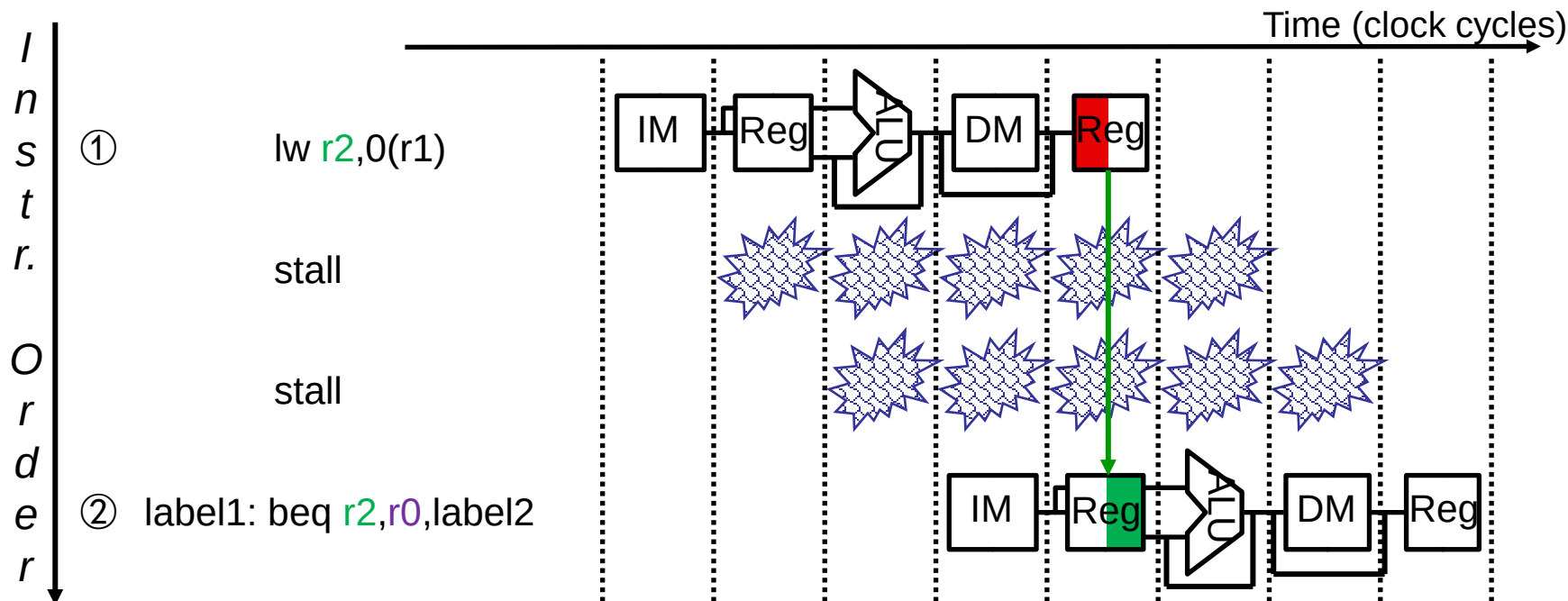
- 如果仅考虑如下的汇编代码段，分支指令在ID阶段提前判断分支（不考虑提前预测），以下说法**错误**的是：

- ① lw r2,0(r1)
- ② label1: beq r2,r0,label2
- ③ add r3, r2, r1
- ④ beq r3,r0,label1
- ⑤ add r1,r3,r1
- ⑥ label2: sw r1,0(r2)

- A. 需要增加EX/MEM到ID的转发（例如3与4之间需要转发）
- B. 不需要增加MEM/WB到ID的转发（Regfile 先写入后读取）
- C. 1与2之间需要stall 1个clock cycle（需要至少2个）
- D. 3与4之间需要stall 1个clock cycle（需要至少1个）

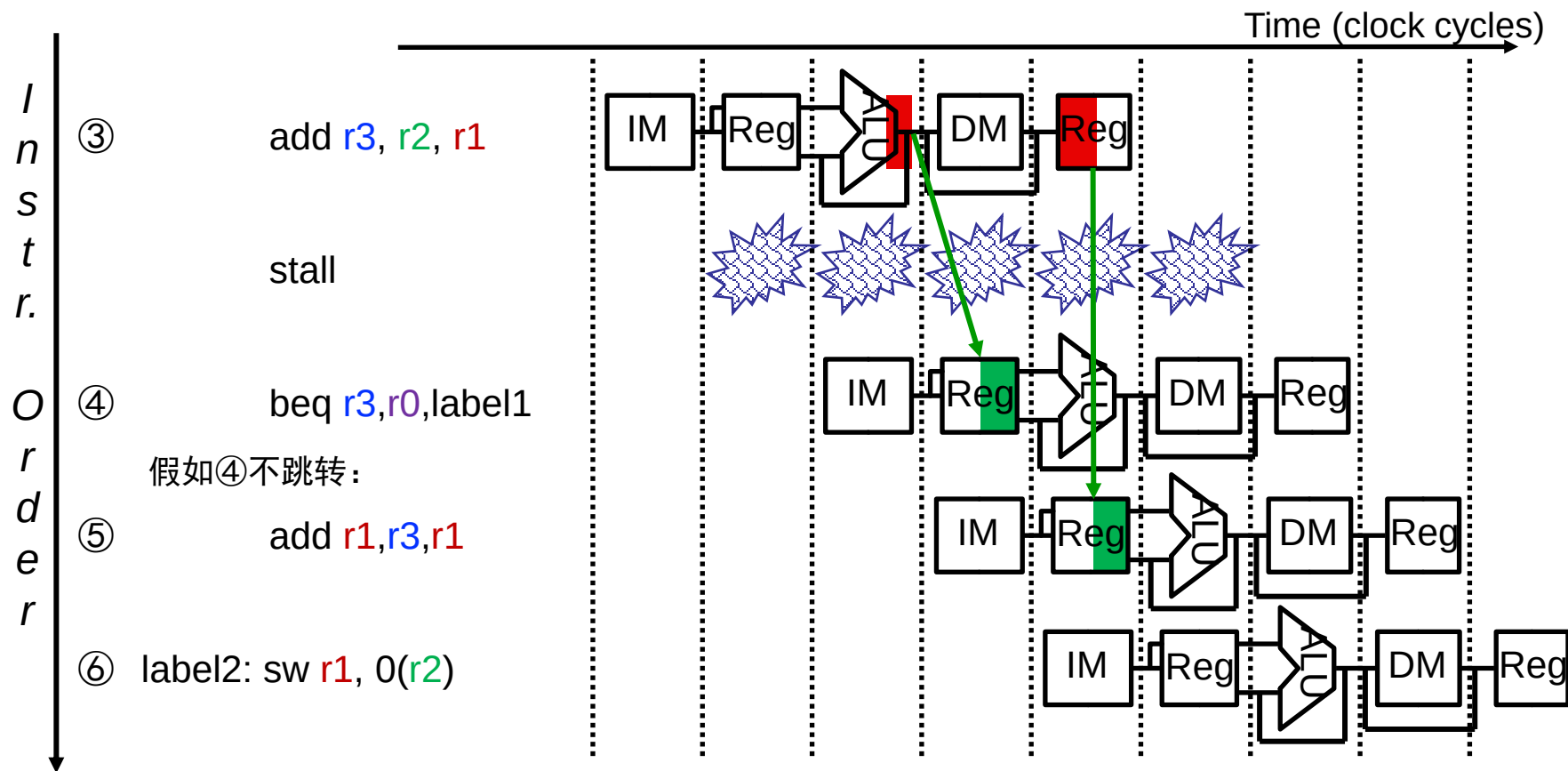
解析：流水线中的各种冒险并不能完全独立，实际情况中，各种冒险可能会相互影响。

答案解析



- A. 需要增加EX/MEM到ID的转发（例如3与4之间需要转发）
- B. 不需要增加MEM/WB到ID的转发（Regfile 先写后读）
- C. 1与2之间需要stall 1个clock cycle（需要至少2个）
- D. 3与4之间需要stall 1个clock cycle（需要至少1个）

答案解析



- A. 需要增加EX/MEM到ID的转发（例如3与4之间需要转发）
- B. 不需要增加MEM/WB到ID的转发（Regfile 先写后读）
- C. 1与2之间需要stall 1个clock cycle（需要至少2个）
- D. 3与4之间需要stall 1个clock cycle（需要至少1个）

另一种控制冒险

- 异常和中断
 - 异常和中断会改变程序的执行流程
- 假设指令 `add r1, r2, r3` 发生了溢出，ALU发现溢出时已经到EX段，后面两条指令已经进了流水线
 - 清除add指令及后面的所有已在流水线中的指令
 - 保存PC或PC+4到EPC
 - 从异常处理程序的首地址（例如0x80000180）处开始取指令

回顾：MIPS的异常处理

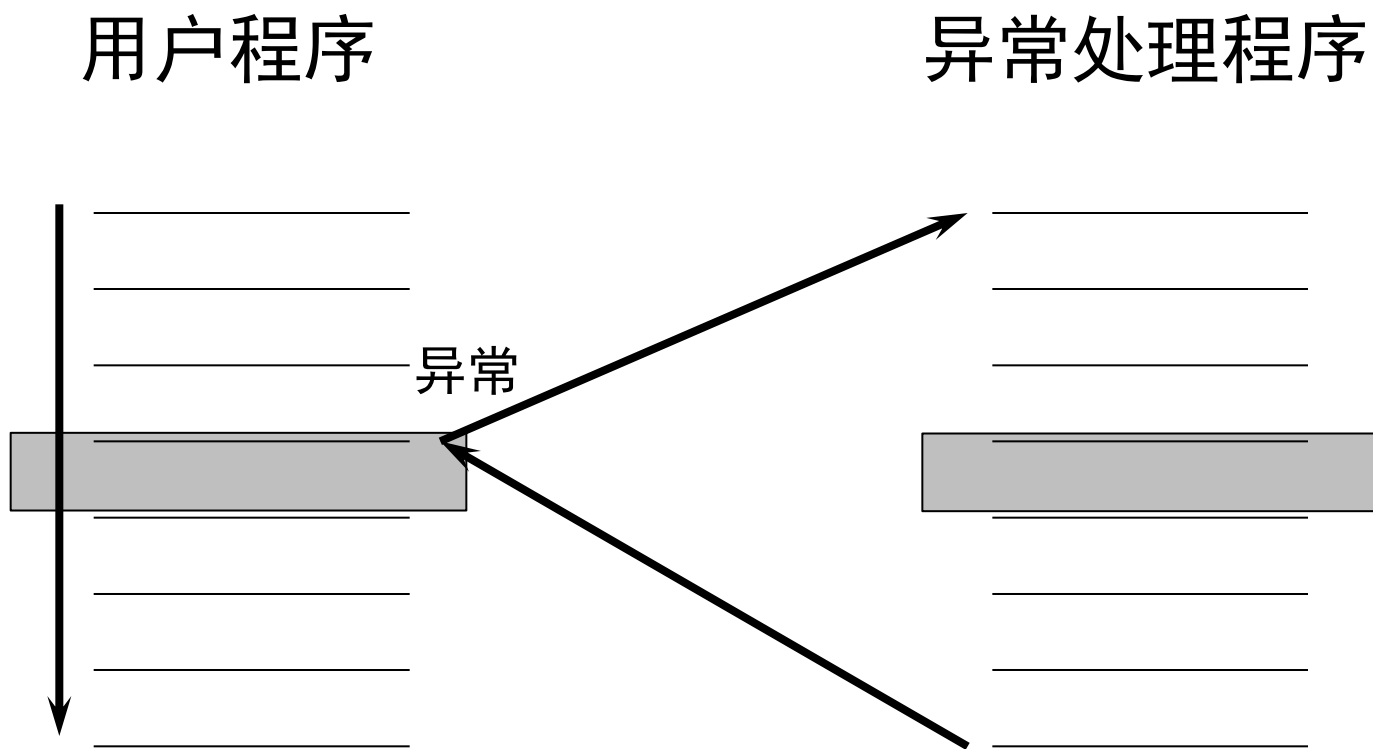
- MIPS约定：异常意味着任何改变控制流的不可预知的事件，它不区分外部和内部
- 当外部原因导致的事件出现时，才使用术语“中断”

事件类型	来源	MIPS指令集术语
用户程序调用	内部	异常
算术溢出	内部	异常
使用未定义指令	内部	异常
硬件故障	外部/内部	异常/中断
I/O设备请求	外部	中断

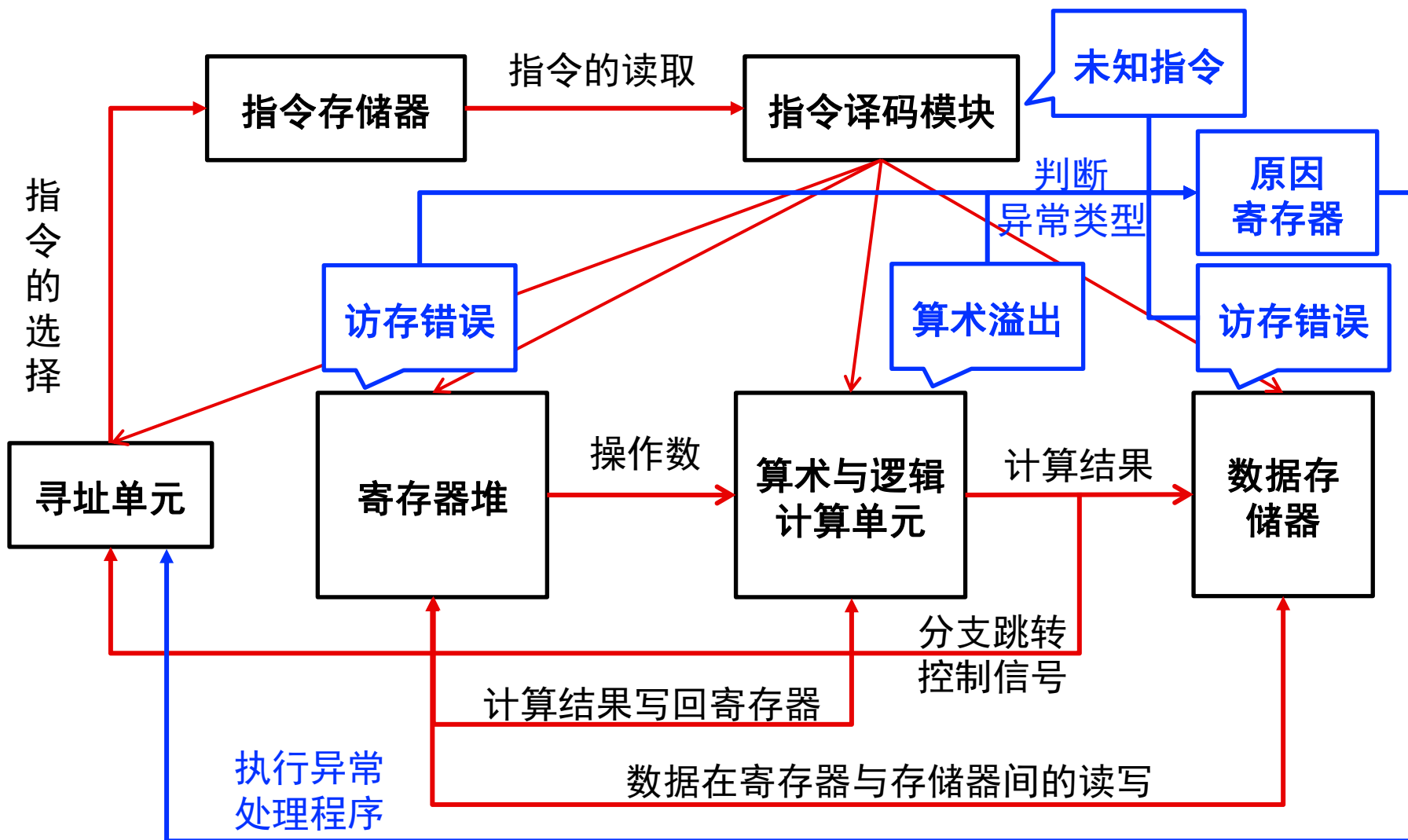
回顾：异常处理

- 正常控制流——顺序执行、分支、跳转、过程调用/返回
 - 异常是非程序控制的控制转移
 - 系统要实施一定动作来处理异常
 - 必须记录下来产生异常的指令的地址
 - 必须保存和恢复用户程序状态
 - 异常处理结束，将控制交还给用户程序
- 

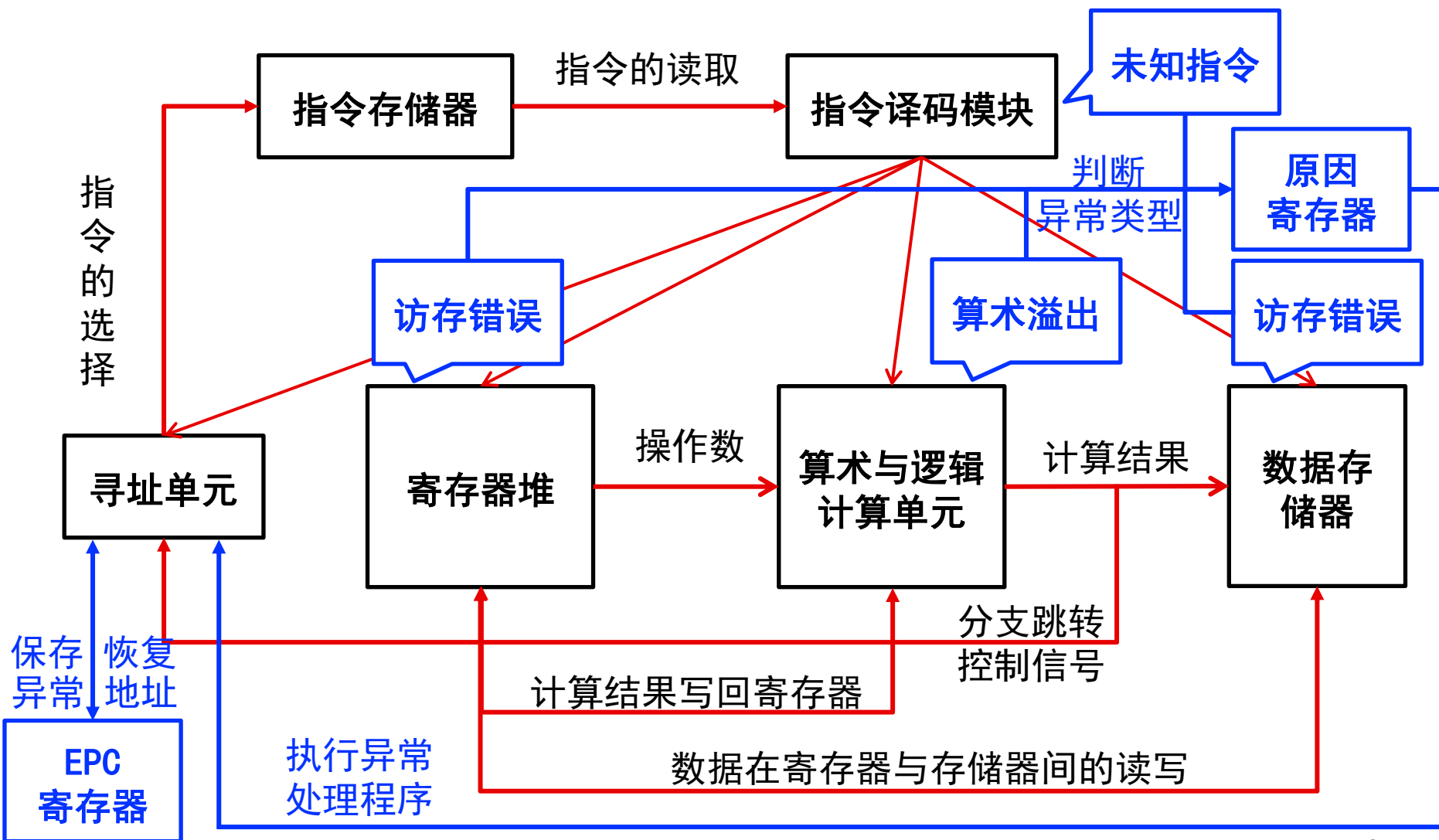
回顾：异常处理



根据异常类型执行异常处理程序

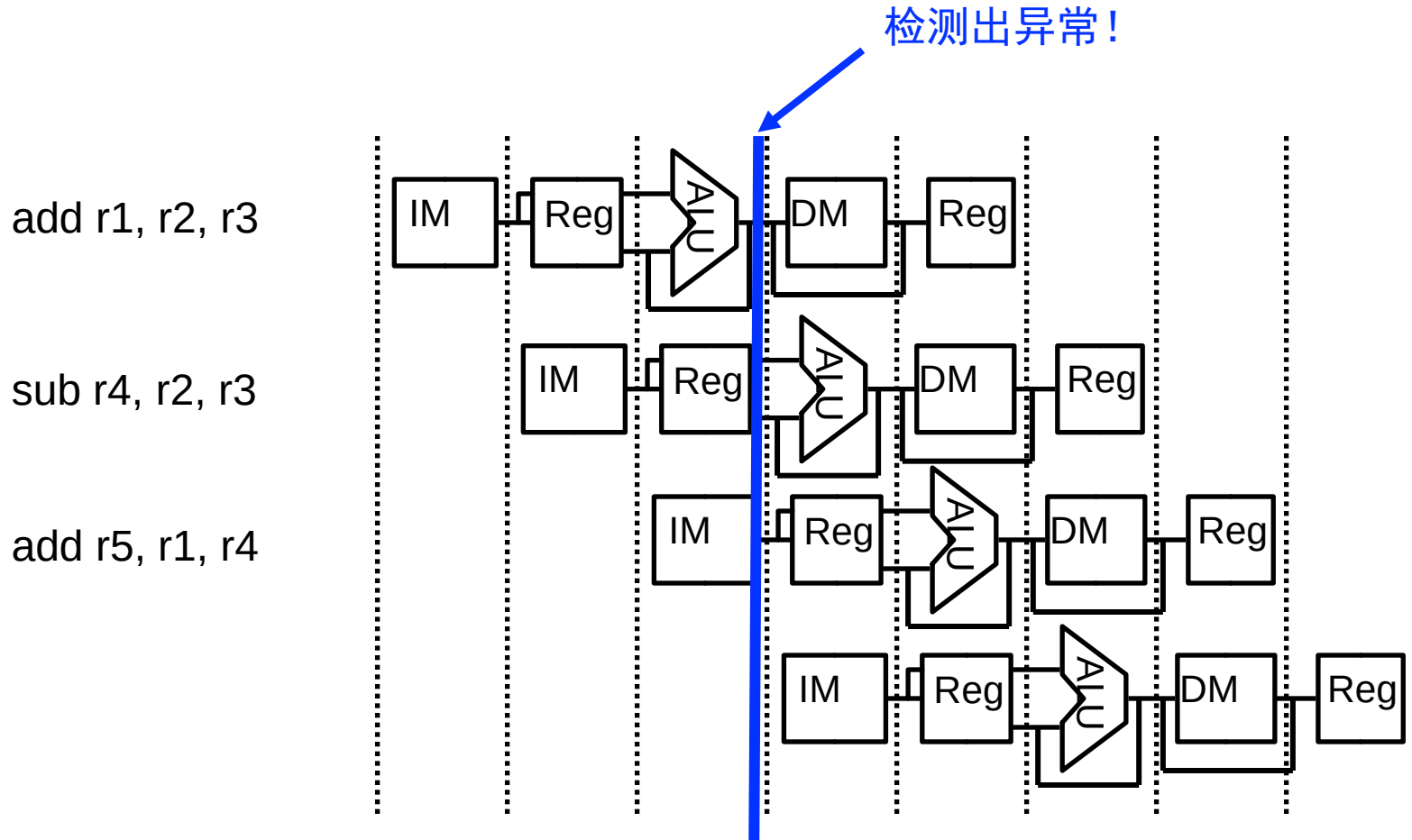


保存/恢复异常指令地址



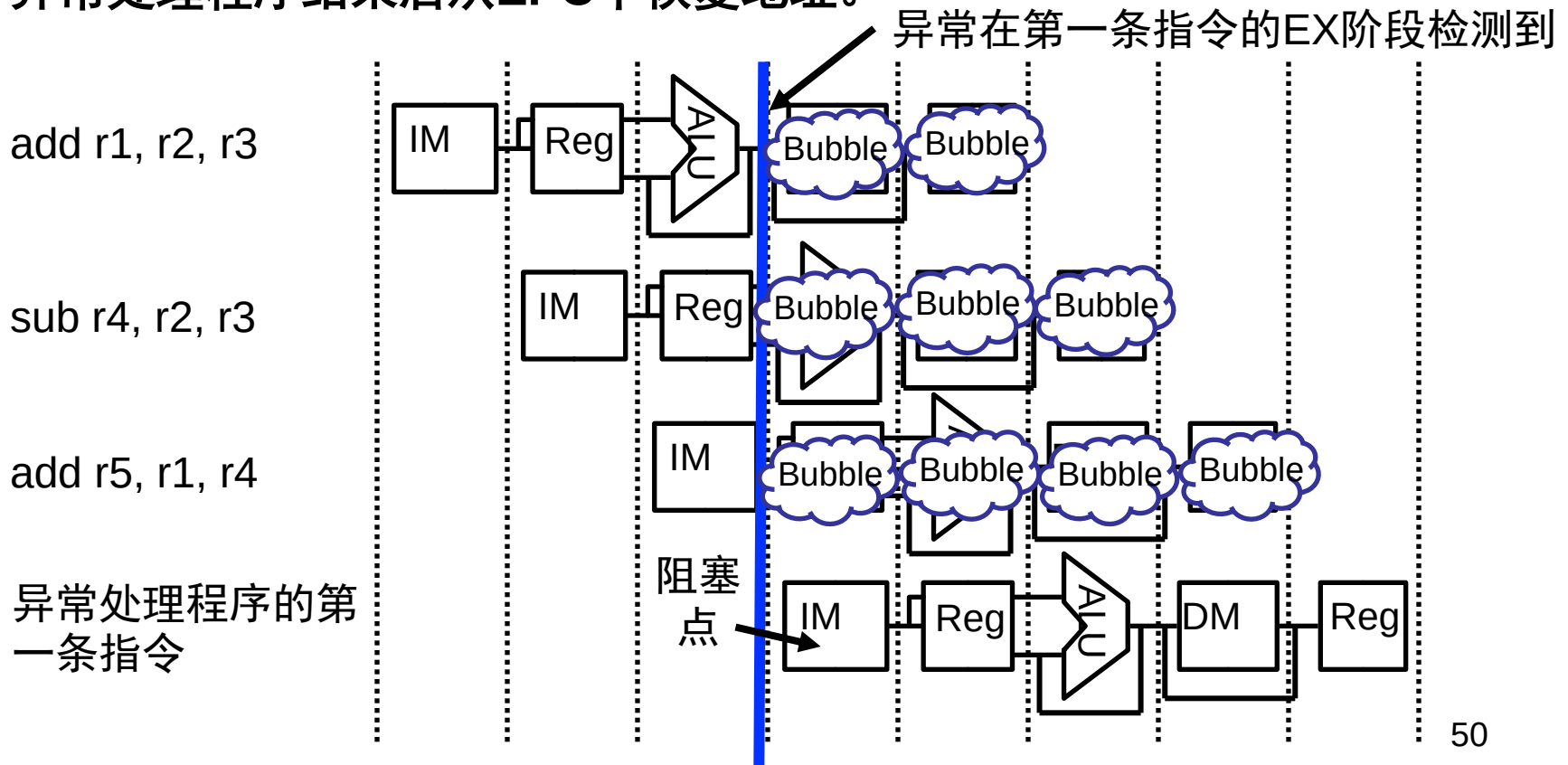
异常处理

- 下图中：异常在第一条指令EX阶段检测出来



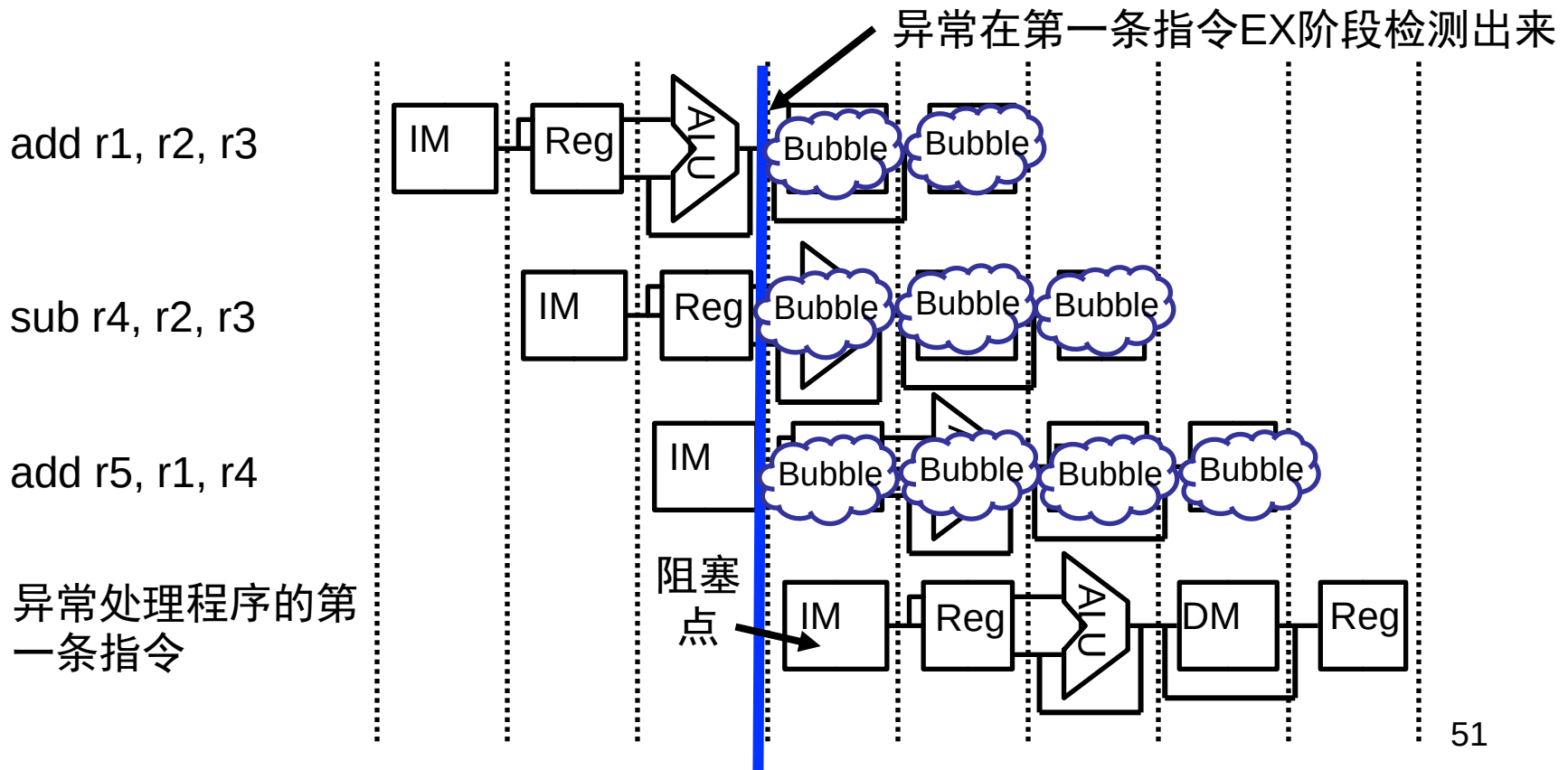
异常处理

- Flush: IF, ID, EX 阶段的控制指令全部清0
- 将异常处理程序的首地址处的指令作为PC输入（通过多路选择器实现）
- 将阻塞点处地址（PC或PC+4）保存到EPC（Exception PC）寄存器。
- 异常处理程序结束后从EPC中恢复地址。

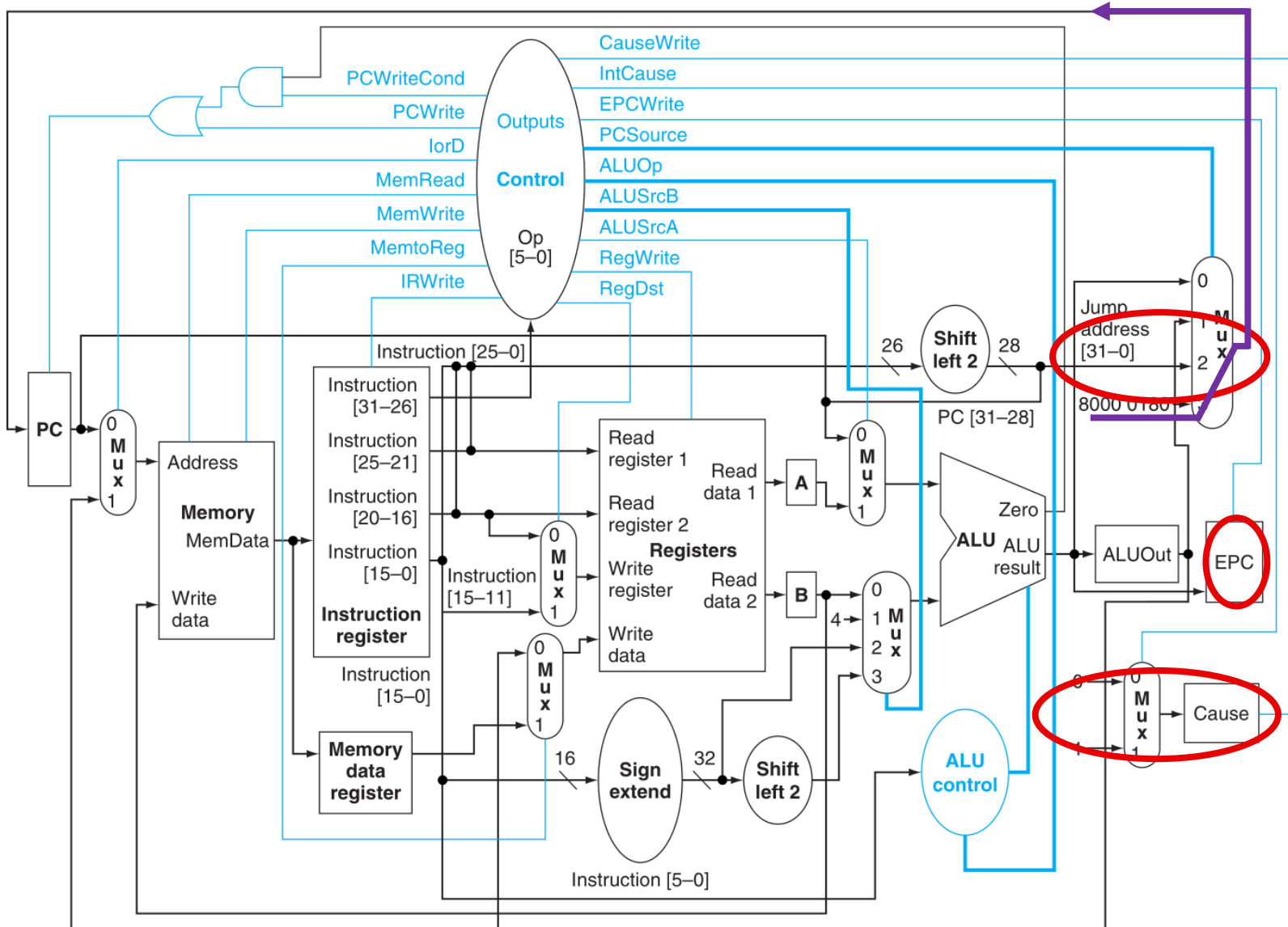


异常处理

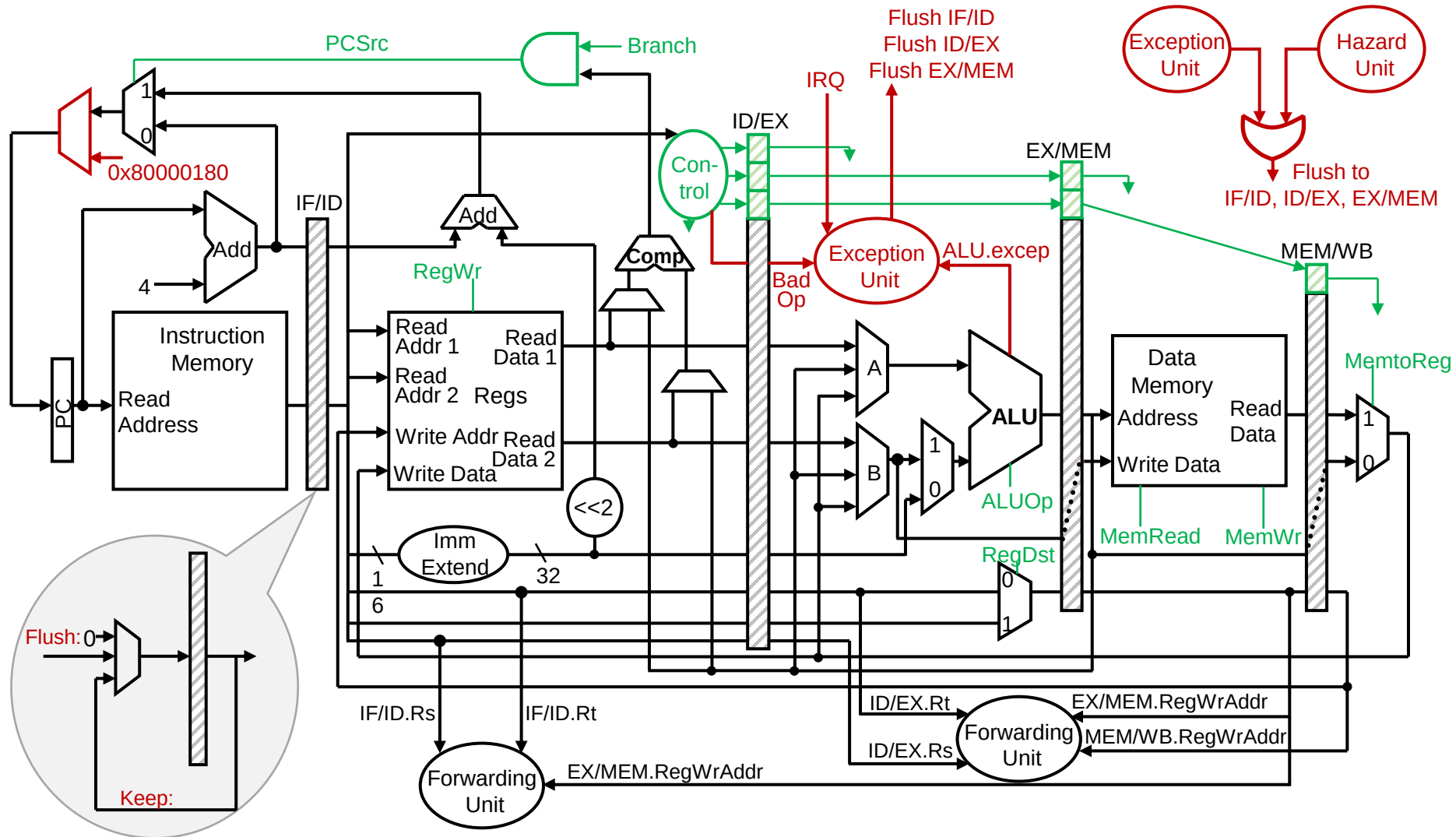
- 是否会将溢出结果写到 r1 寄存器？
 - 不会！EX.flush信号使得EX阶段指令控制信号清零（RegWr置为0），因此WB阶段不会发生寄存器写入



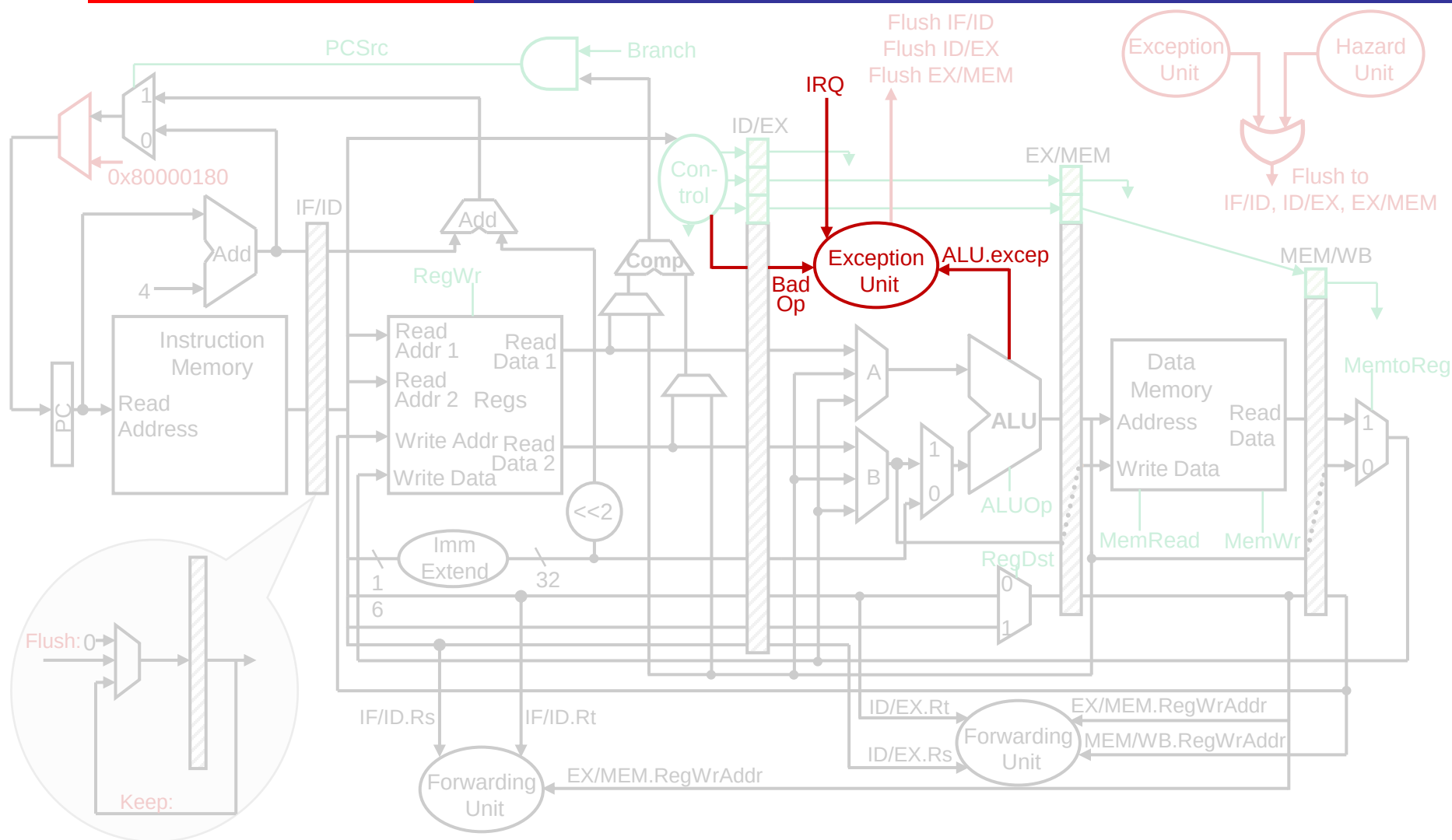
多周期处理器的异常处理



添加异常处理单元

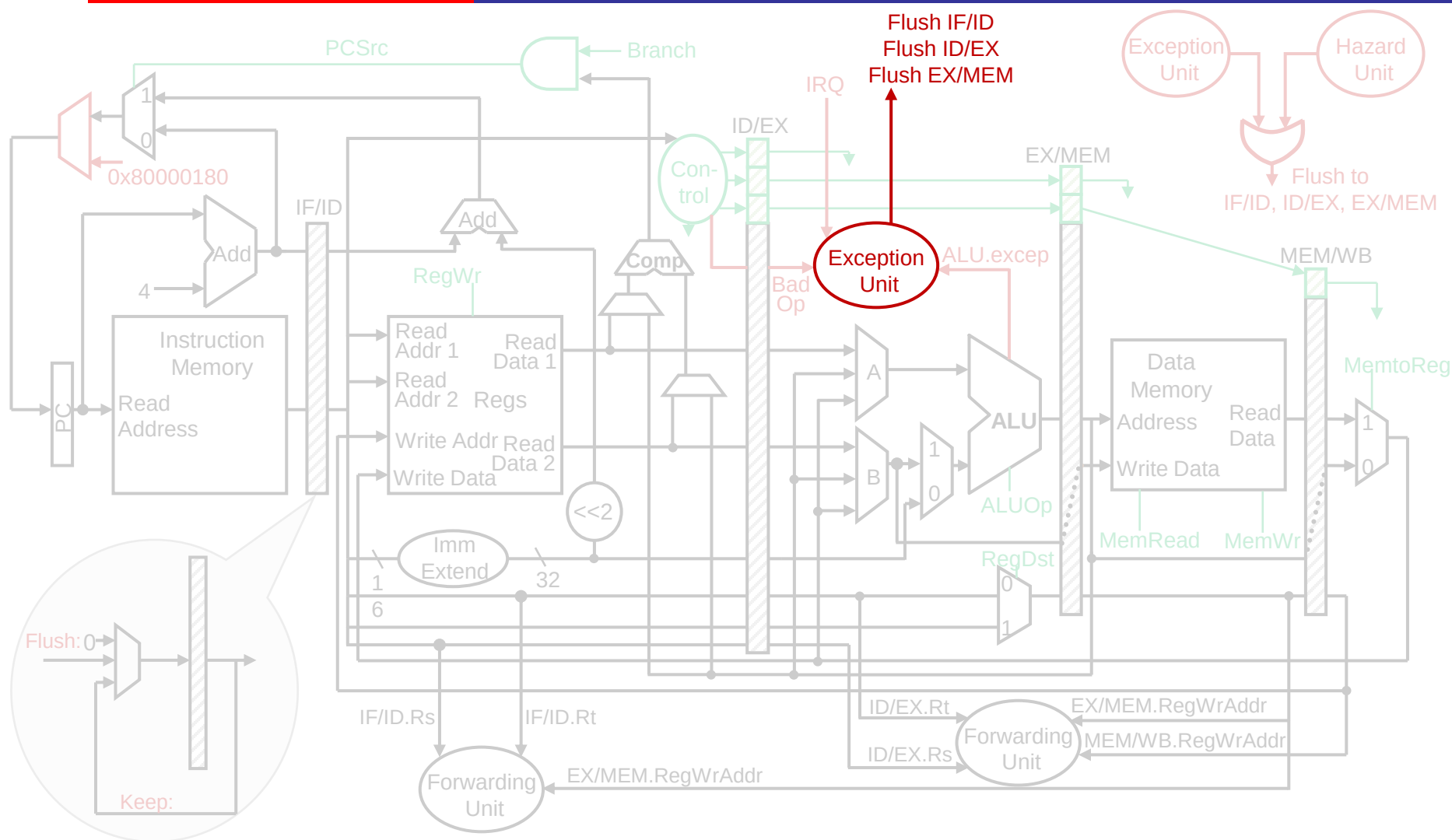


添加异常处理单元



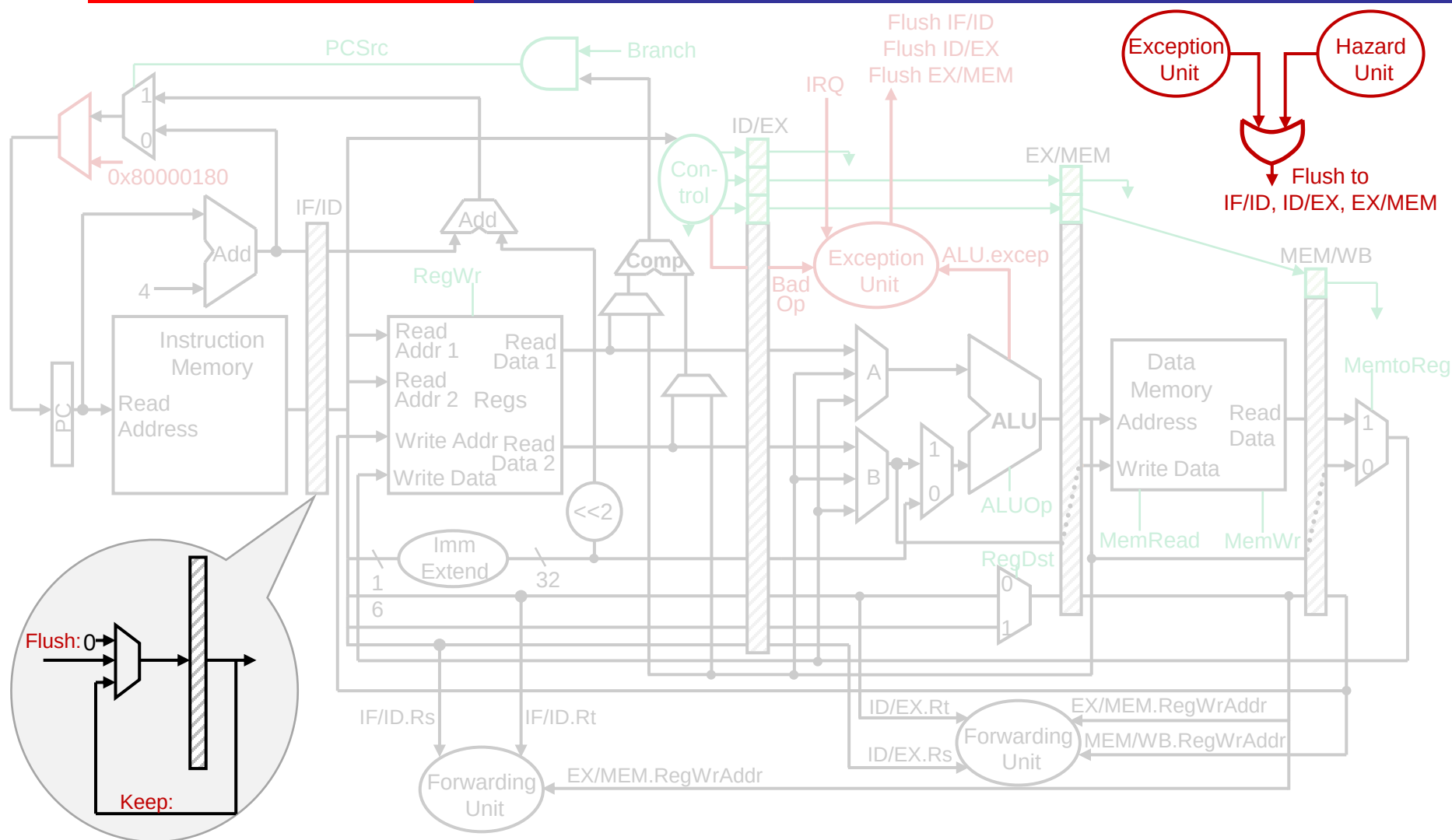
- 异常处理单元输入: BadOp, ALU异常 (溢出), IRQ

添加异常处理单元



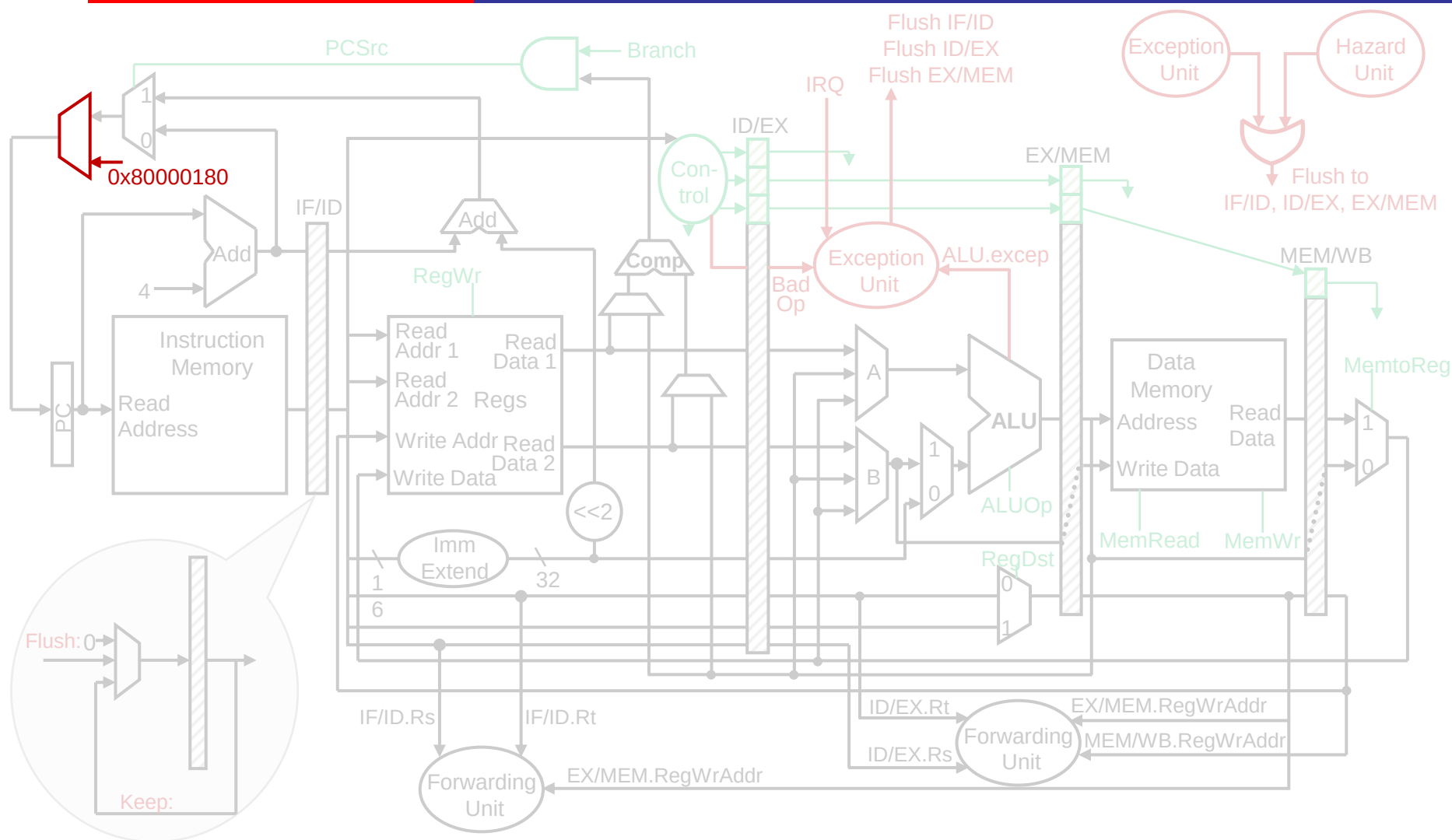
- 异常处理单元输出: Flush IF/ID, Flush ID/EX, Flush EX/MEM 55

添加异常处理单元



- Flush信号加入或门，使得Exception单元和Hazard单元同时控制Flush

添加异常处理单元



- 地址选择处添加异常处理程序首地址及选择器控制信号

流水线结构与冒险小结

- 流水线结构
 - 5级经典方式：IF ID EXE MEM WB
 - 流水线作用（理论与实际的吞吐率，提升潜力：级数）、代价（单条指令延迟）、时钟频率瓶颈（最慢的一级）
- 冒险及应对策略
 - 结构冒险：REG W&R、RF多总线、MEM、ALU分离
 - 数据冒险：Forwarding、Stall、编译调度
 - 控制冒险：Stall、Forwarding、延迟槽、分支预测
- 要求
 - 理解冒险来源（会检测与区分冒险）
 - 掌握应对策略的思想（数据与结构的关联性）
 - 熟悉数据通路的具体工作方式（理解数据流变动）

本节课目录

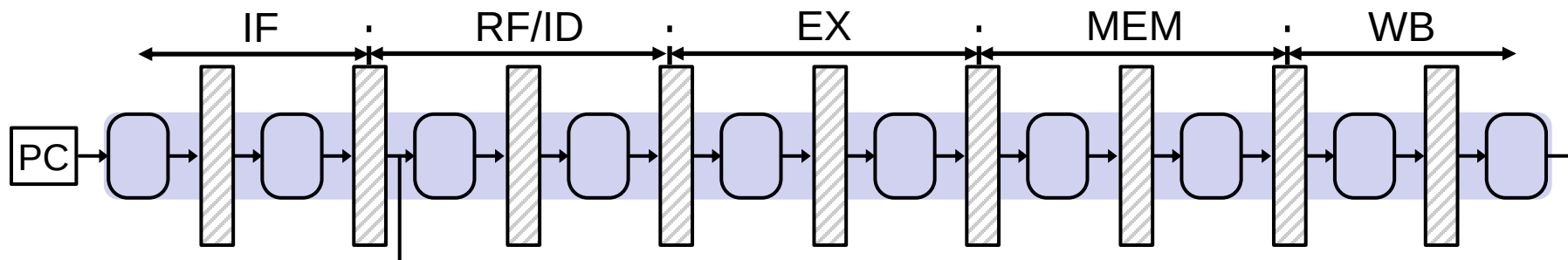
- 数据通路的实现（考虑冒险） P12
- 进一步提升处理器性能
 - 指令级并行 P60
 - 超长指令字
 - 超标量
 - 线程级并行 P63
 - 超线程
 - 多核
 - 异构计算 P66
 - GPU
 - XPU

指令级并行

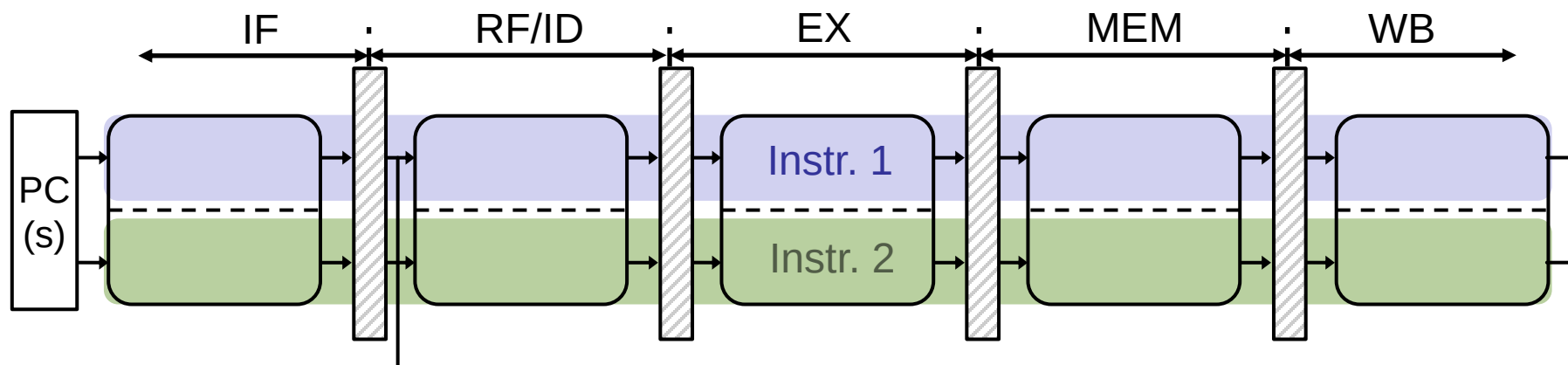
- CPU性能
 - 执行时间=指令数 $\times$ CPI $\times$ 时钟周期
- 以普通流水线处理器性能为例
 - 最好情况的CPI: 接近为1
 - 最短时钟周期: 约为单周期处理器时钟周期的1/5
- 如何进一步提高性能?
 - 指令级并行（并行执行多条指令）
 - 缩短时钟周期: 超级流水线
 - 增加流水线深度
 - 降低CPI: 多发射
 - 每个周期取指和执行多条指令

超级流水线 vs 多发射

- 超级流水线：更深的流水线



- 多发射：更宽的流水线

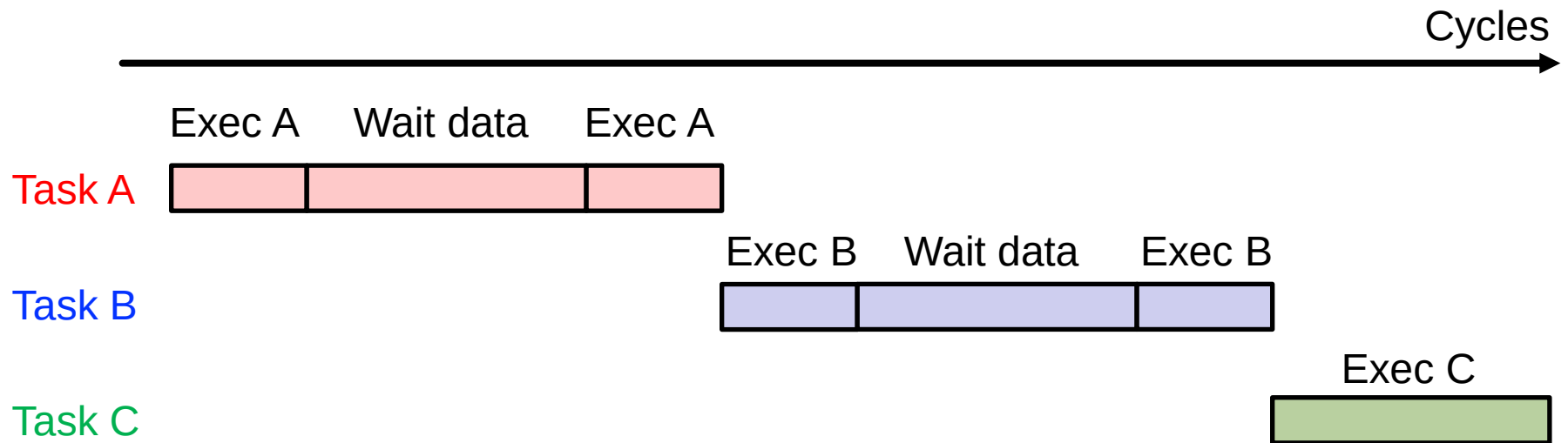


本节课目录

- 数据通路的实现（考虑冒险） P12
- 进一步提升处理器性能
 - 指令级并行 P60
 - 超长指令字
 - 超标量
 - 线程级并行 P63
 - 超线程
 - 多核
 - 异构计算 P66
 - GPU
 - XPU

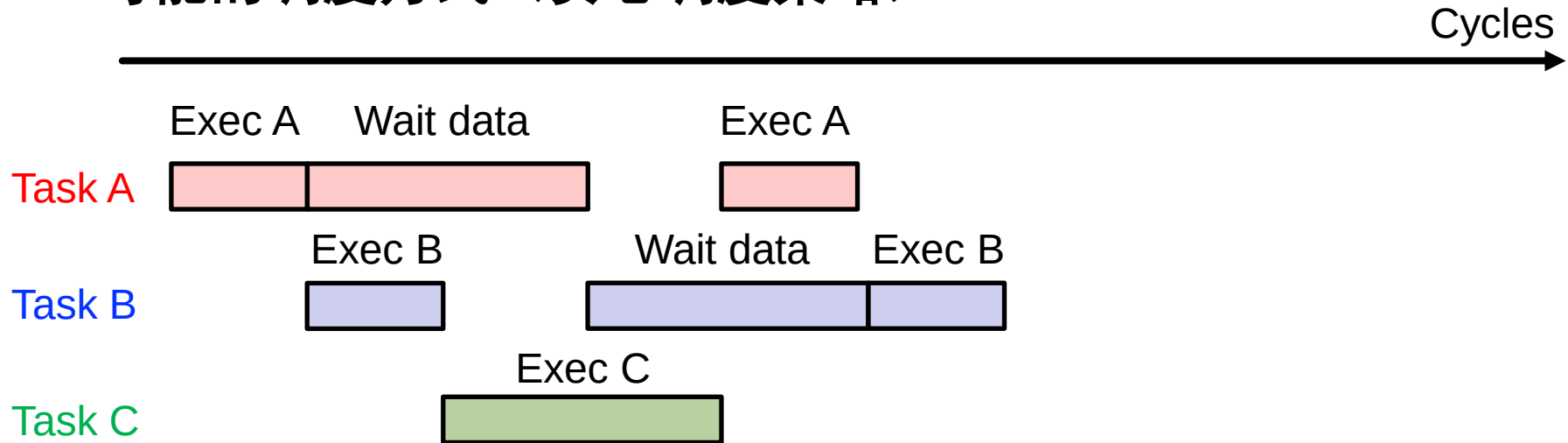
超线程

- 在实际CPU执行过程中，由于访存时间较长（通常需要多个周期），经常会遇到等待访存的情况
- 解决方案？
 - 等待时执行其他任务



超线程

- 可能的调度方式（贪心调度策略）



- 充分利用资源
 - 利用空闲资源执行其他任务
 - 在外部看起来就像CPU在同时执行多个任务

本节课目录

- 数据通路的实现（考虑冒险） P12
- 进一步提升处理器性能
 - 指令级并行 P60
 - 超长指令字
 - 超标量
 - 线程级并行 P63
 - 超线程
 - 多核
 - 异构计算 P66
 - GPU
 - XPU

GPU

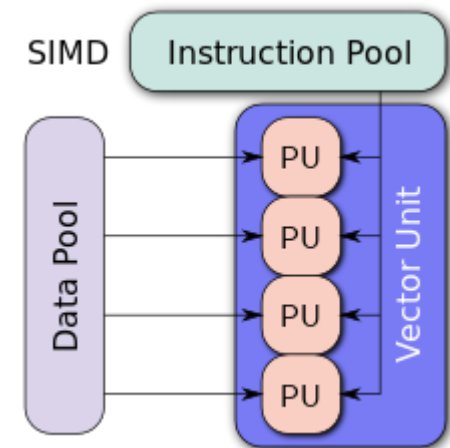
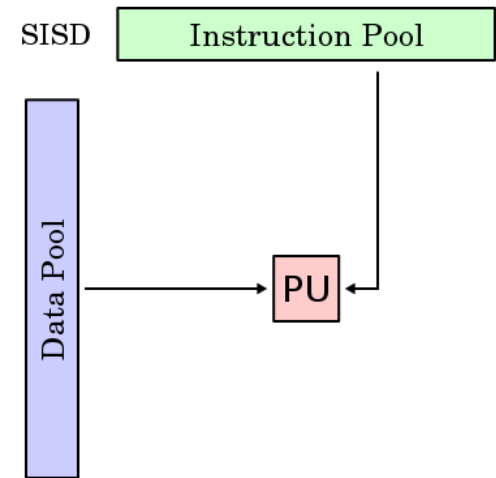
• CPU vs GPU

– CPU (SISD)

- Single Instruction Single Data (SISD)
- 单指令流单数据流
 - ✓ 并行度较小
 - ✓ 灵活度高
- 现代CPU可以通过SSE、AVX等扩展指令集支持SIMD
 - SSE: Streaming SIMD Extensions
 - AVX: Advanced Vector Extensions

– GPU (SIMD)

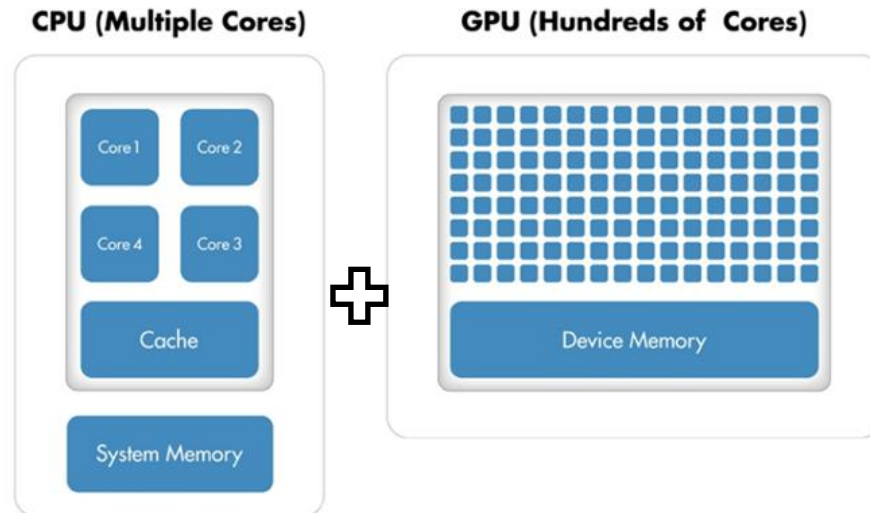
- Single Instruction Multiple Data
- 单指令流多数据流
 - ✓ 具有更大的并行度
 - ✓ 设计相对比较简单



GPU

- 异构计算

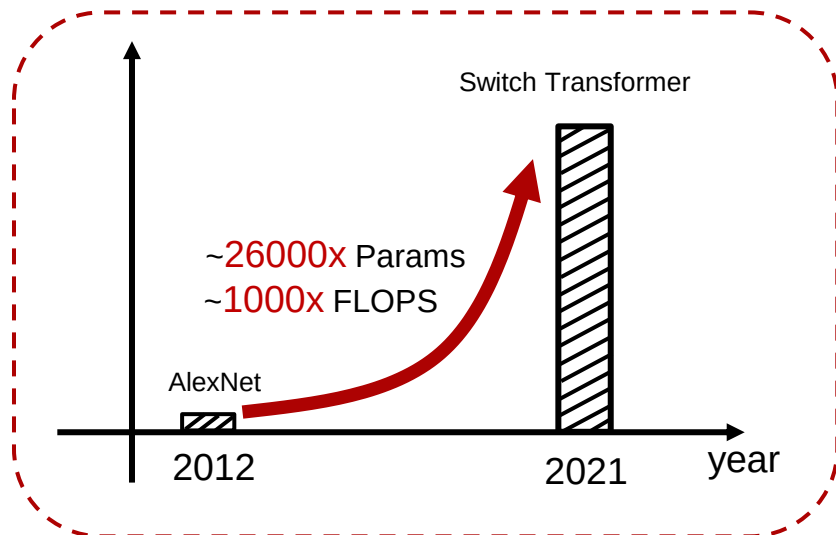
- 根据计算任务特性不同，分配到不同的硬件去执行，最大化系统性能
- 把串行执行的程序放在擅长低延时的处理器（CPU）上
- 把结构简单，能并行执行的程序放在擅长大吞吐量的处理器（GPU）上



本节课目录

- 数据通路的实现（考虑冒险） P12
- 进一步提升处理器性能
 - 指令级并行 P60
 - 超长指令字
 - 超标量
 - 线程级并行 P63
 - 超线程
 - 多核
 - 异构计算 P66
 - GPU
 - XPU

智能计算面临的矛盾



智能算法算力要求**激增**



通用计算硬件能效提升**缓慢**

1. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25, 1097-1105.
- 2 Fedus, W., Zoph, B., & Shazeer, N. (2021). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv preprint arXiv:2101.03961.

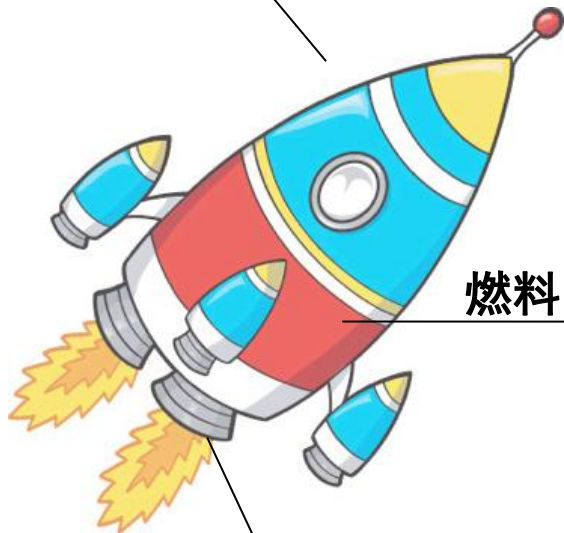
XPU

- 针对不同类型的任务，设计面向专用领域的处理器XPU，提升能效

– 以深度神经网络计算领域为例

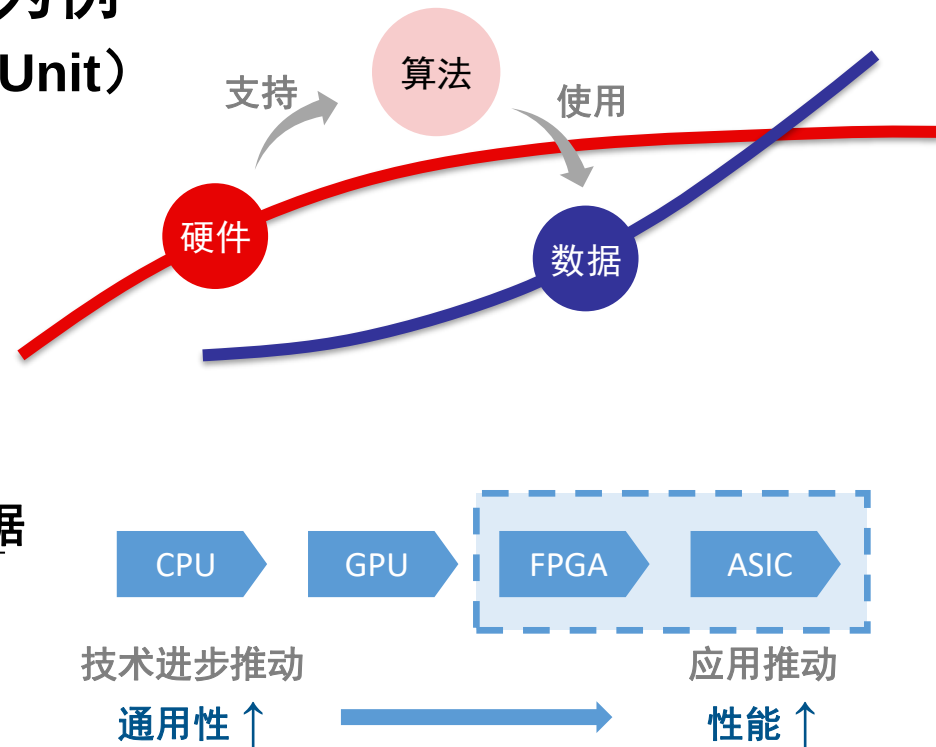
- NPU（Neural Processing Unit）

火箭：深度学习



燃料：大数据

引擎：计算平台



拓展内容小结

- 进一步提升处理器性能的方式
 - 指令级并行
 - 超标量、超长指令字：基本原理、区别、对CPI的影响
 - 线程级并行
 - 超线程：可以提高利用率
 - 多核：并行、数据同步
 - 异构计算
 - GPU：SIMD、与CPU的区别
 - XPU：定制加速器思路、高能效比
- 能力
 - 了解基本原理和设计思想的转变即可
 - 不重点考察，可能会考察基本原理，不会考察相关计算和具体细节

作业

- 要求
 - 上传电子版至网络学堂
 - 不要抄袭或迟交
- 流水线作业-3
 - 教材第7章课后习题8(1)(2), 13
 - 题目以网络学堂作业附件为准

指令的执行过程对比

- 超级流水线：更深的流水线

IF1	IF2	ID1	ID2	EX1	EX2	MEM1	MEM2	WB1	WB2			
	IF1	IF2	ID1	ID2	EX1	EX2	MEM1	MEM2	WB1	WB2		
		IF1	IF2	ID1	ID2	EX1	EX2	MEM1	MEM2	WB1	WB2	
			IF1	IF2	ID1	ID2	EX1	EX2	MEM1	MEM2	WB1	WB2

- 多发射：更宽的流水线

静态多发射（超长指令字）

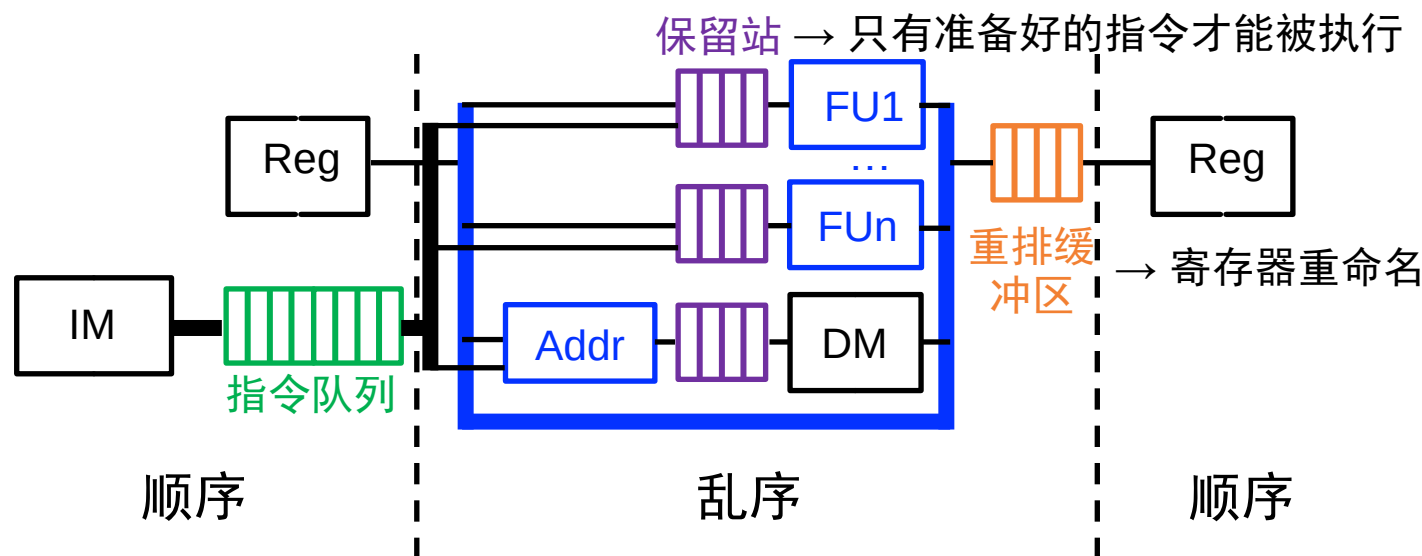
IF	ID	EX	MEM	WB	
		EX	MEM	WB	
	IF	ID	EX	MEM	WB
			EX	MEM	WB

动态多发射（超标量）

IF	ID	EX	MEM	WB	
IF	ID	EX	MEM	WB	
	IF	ID	EX	MEM	WB
	IF	ID	EX	MEM	WB

后续课程

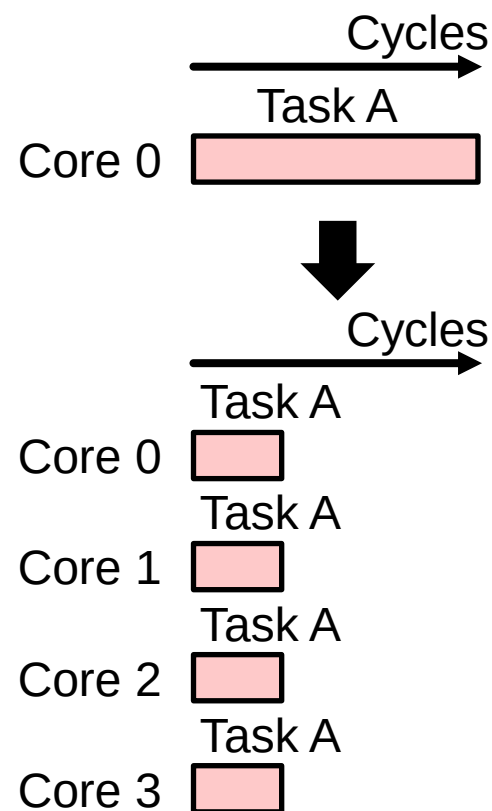
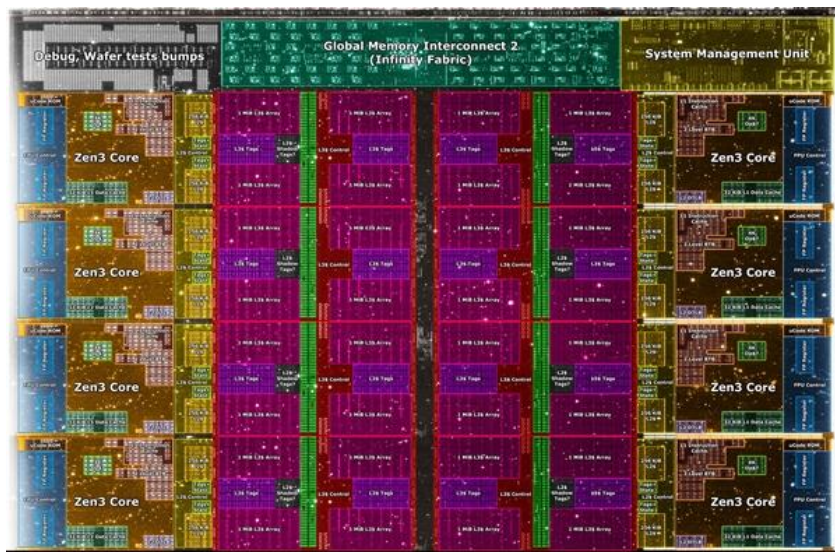
- 超标量处理器中还包括若干其他技术
 - 乱序执行、解决更加复杂的冒险、指令队列、保留站、寄存器重命名、硬件推测、循环展开、计分板、动态内存消歧等
- 具体实现可以在后续课程中学习
 - 电子系-刘勇攀老师-现代计算机体系架构



多核处理器

- 现代处理器大多具有很多个处理器核心
- 多个核心可以并行执行任务

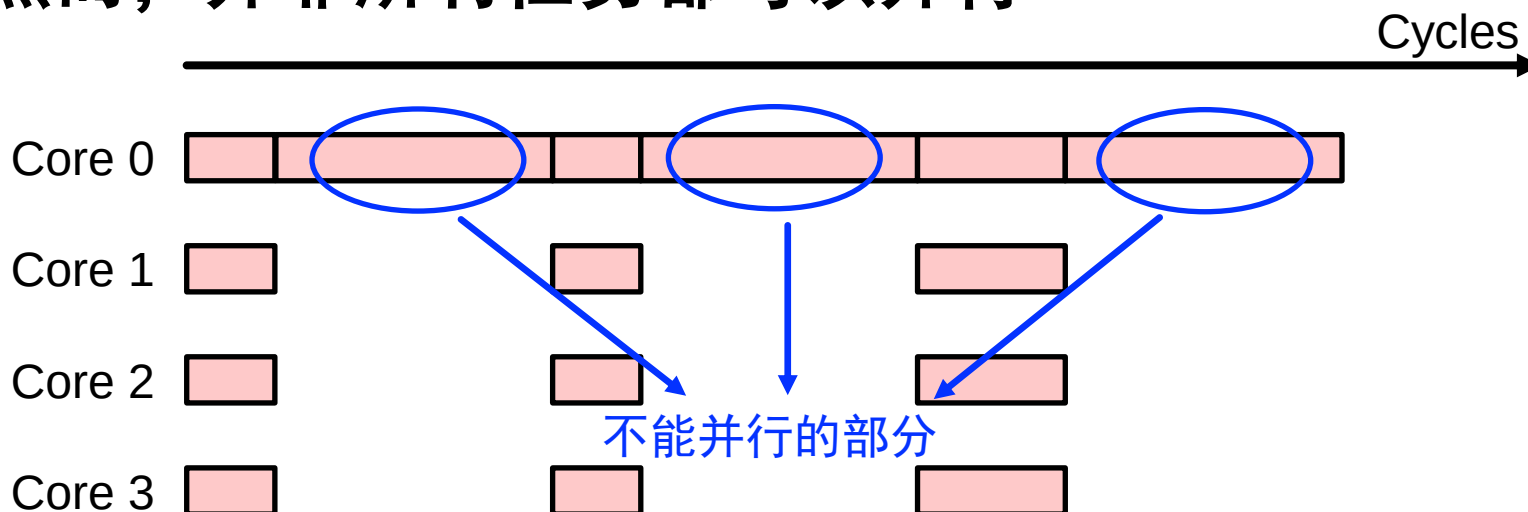
AMD Zen3 8-Core CPU



可能存在的问题？

多核处理器

- 然而，并非所有任务都可以并行



- 阿姆达尔定律 (Amdahl's Law)

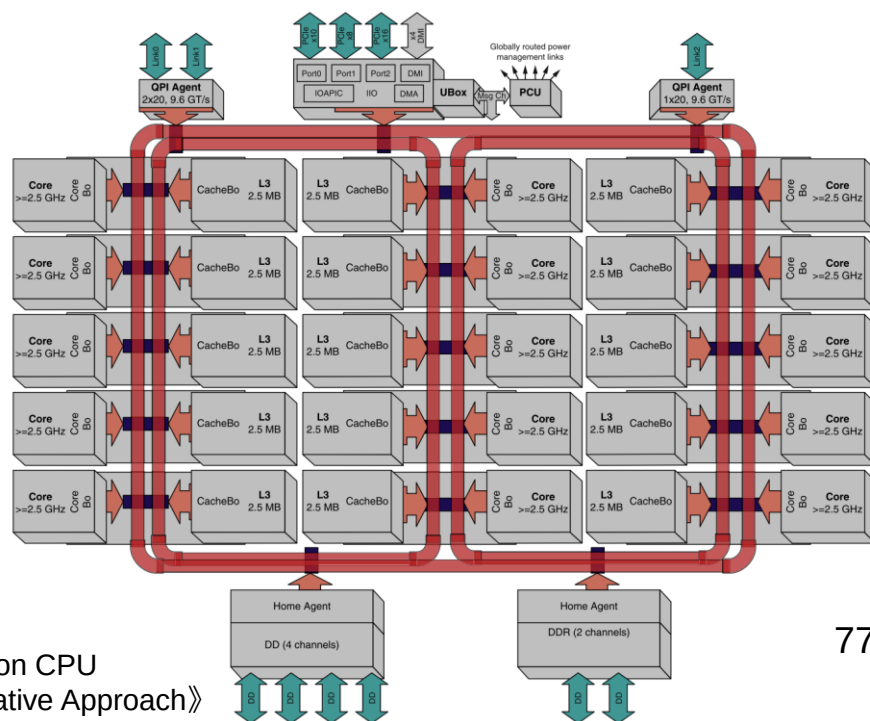
- 并行系统的加速比 $S = \frac{1}{(1-a)+a/n}$

- 其中a为并行部分所占比例，n为并行部分的加速比

- 并行计算的加速比有上限

多核处理器

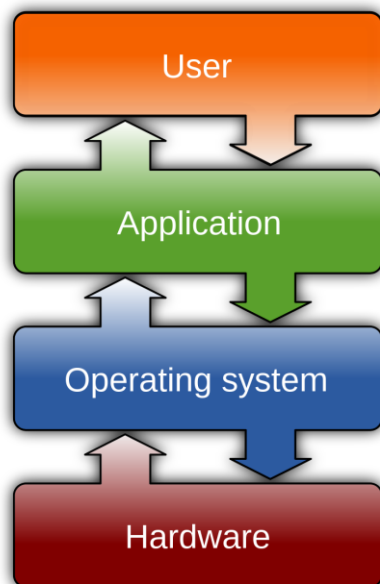
- 核数越多越好吗？
- 阿姆达尔定律（Amdahl's Law）
 - 随着核数增多，增加核数的边际收益降低
- 多核间数据共享
 - 随着核数增多，处理器核与核之间的数据通信越来越复杂
 - 可能采用环形总线等方式进行互联



Core interconnection on Intel E7 Xeon CPU
《Computer Architecture, A Quantitative Approach》

后续课程

- 对于多核、多个线程之间的CPU、内存等硬件资源的分配和调度，可以在后续课程中继续学习
 - 进程、线程、处理机管理、存储管理、文件系统、I/O 管理、设备驱动程序
 - 电子系-马洪兵老师-操作系统



GPU

- **GPU (Graphics Processing Unit)**
 - 为处理2D/3D图像、视频、视觉计算、显示任务专门设计的芯片
 - 1999年，NVIDIA发布GeForce 256时首次提出GPU的概念
 - 2006年，NVIDIA发布GeForce 8800 GPU时提出了CUDA的概念
 - Compute Unified Device Architecture
 - 利用GPU的高性能和高带宽优势，加速通用计算的一些操作
 - GP-GPUs (General-Purpose GPUs)



第一个GPU：
GeForce 256



GPU

- 应用场景

- 个人电脑

- NVIDIA
 - AMD
 - Intel

- 嵌入式系统

- NVIDIA Tegra

- 智能手机、平板

- Qualcomm Snapdragon

- 云端数据中心、超算

- 天河2号

- 32000 Xeon (CPU) + 48000 Xeon Phi (GPU)

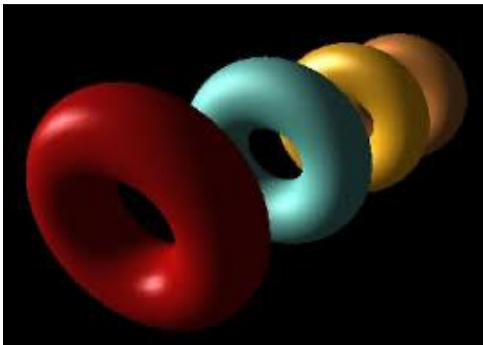
- 峰值算力 54.9 PFLOPs



GPU

• 应用场景

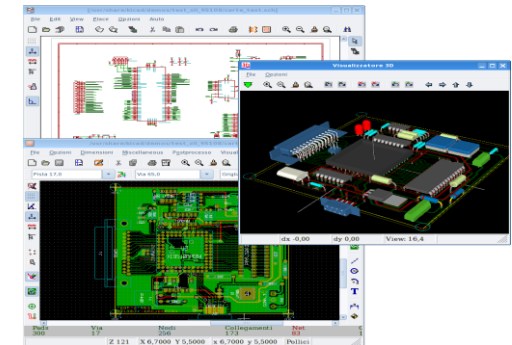
3D图像渲染



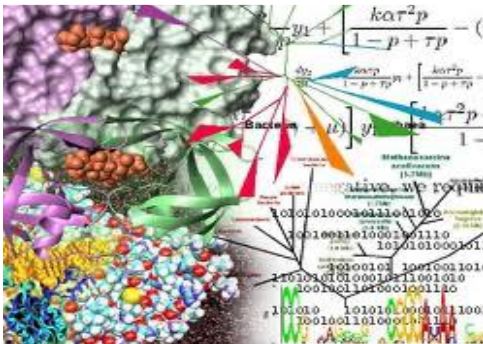
增强现实



电子设计自动化 (EDA)



生物学



金融



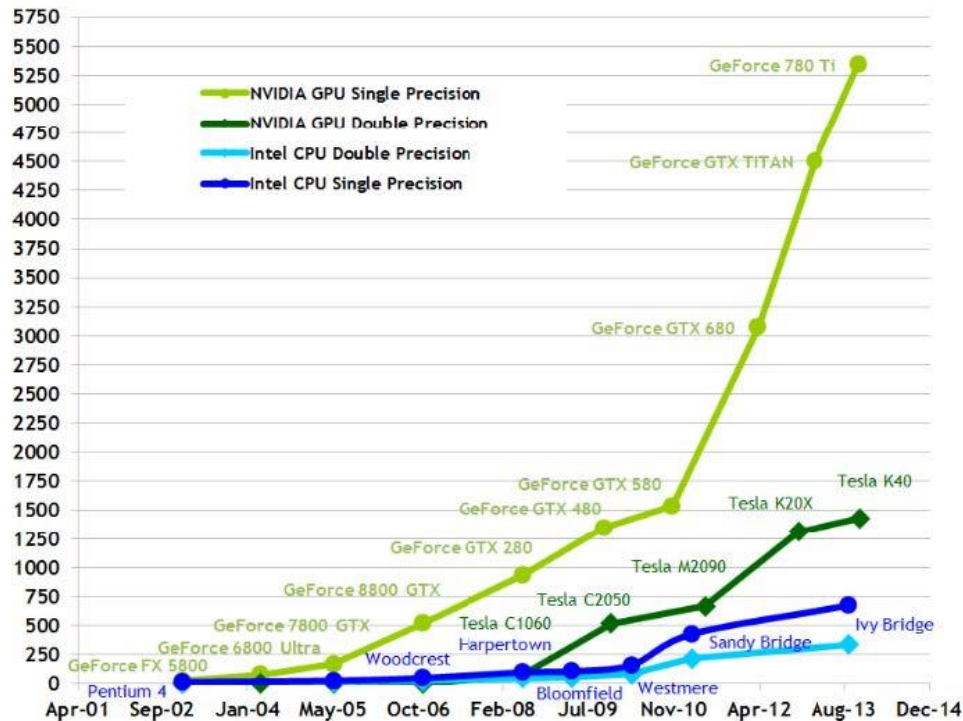
深度学习



GPU

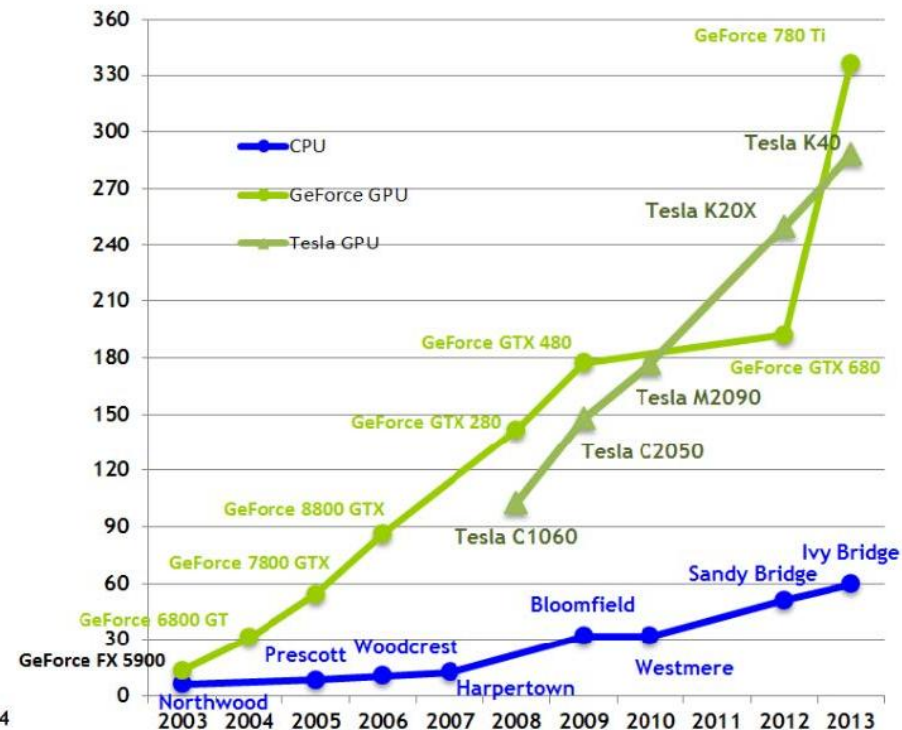
- Why GPU?
– GPU vs CPU

Theoretical GFLOP/s



运算能力 (GFLOP/s)

Theoretical GB/s

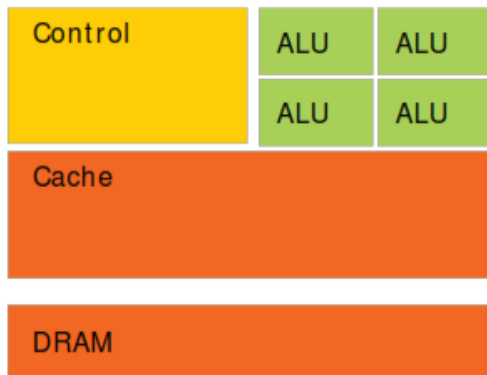


内存带宽 (GB/s)

GPU

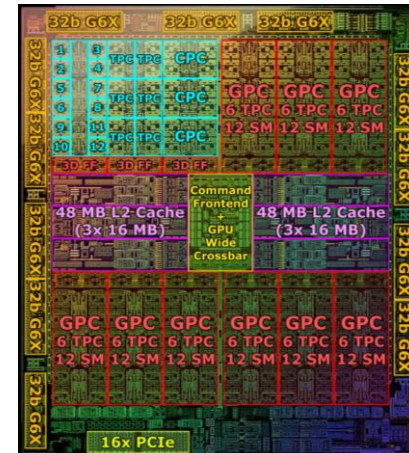
• CPU vs GPU

Intel Core i9 13900K



CPU

NVIDIA AD102 RTX 4090



GPU

GPU

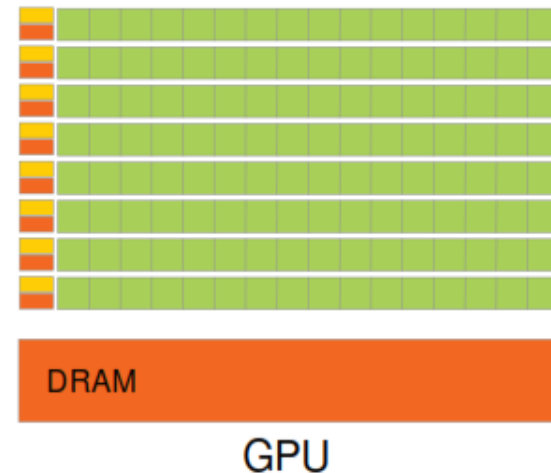
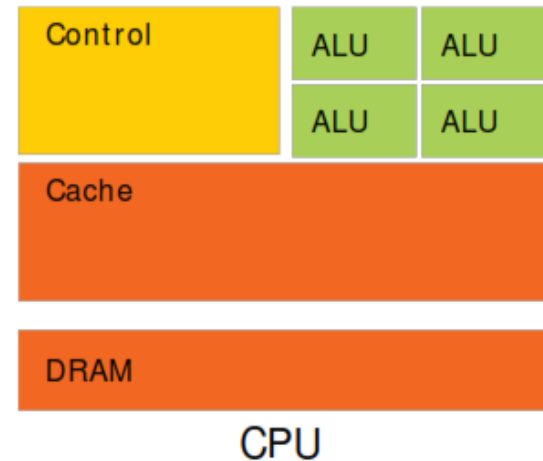
• CPU vs GPU

– CPU

- 复杂的大核，核数较少
- （每个核的）缓存更大
- 控制密集
- 擅长branch, jump
- 粗粒度线程级并行

– GPU

- 简单的小核，核数很多
- （每个核的）缓存很小
- 计算密集
- 对于分支跳转指令性能很差
- 擅长流式执行
- 细粒度数据级并行



GPU

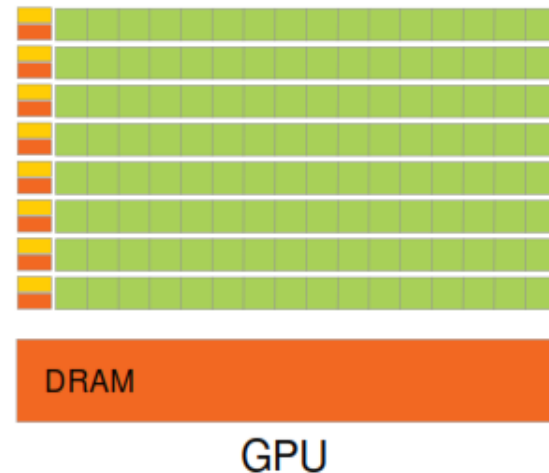
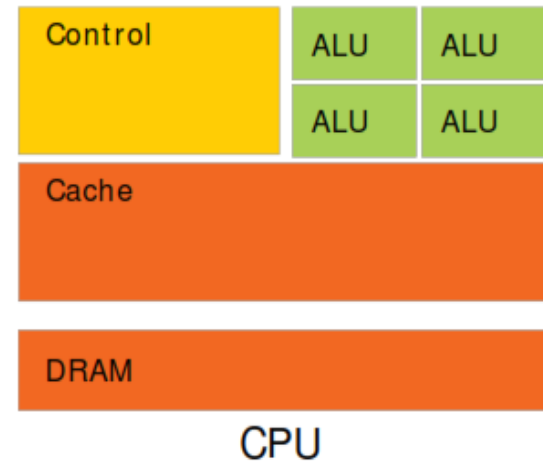
• CPU vs GPU

– CPU

- 带宽：
 - DDR4: 约 50 GB/s (64bit接口)
- 缓存容量：
 - 最后一级缓存每个核约1MB或更多
- 访存延时：
 - DDR4 DRAM: 约 30 cycles

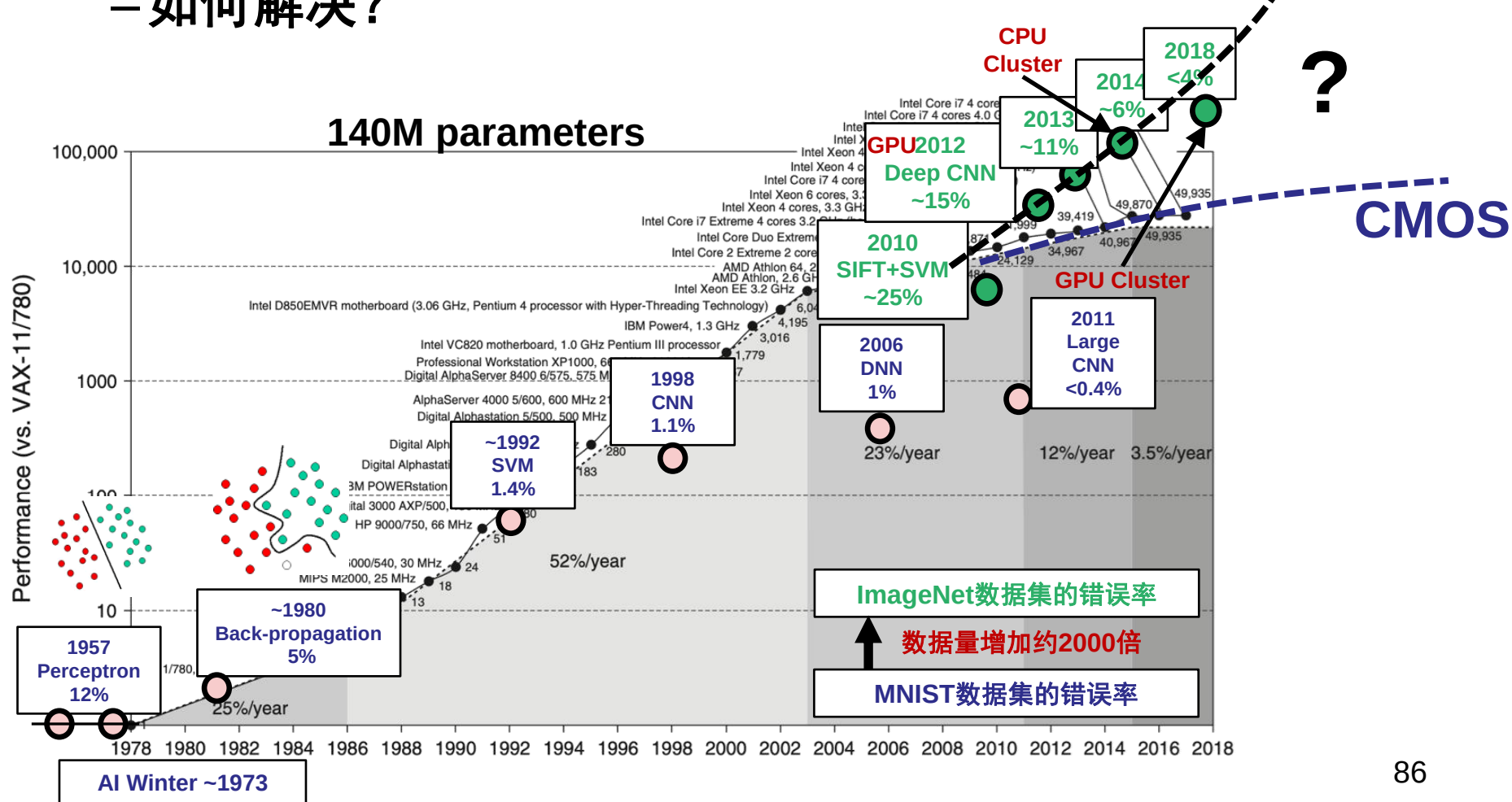
– GPU

- 带宽：
 - GDDR5: > 200 GB/s (384bit接口)
- 缓存容量：
 - 最后一级缓存一组核约1MB
- 访存延时：
 - GDDR5: 约 500 cycles

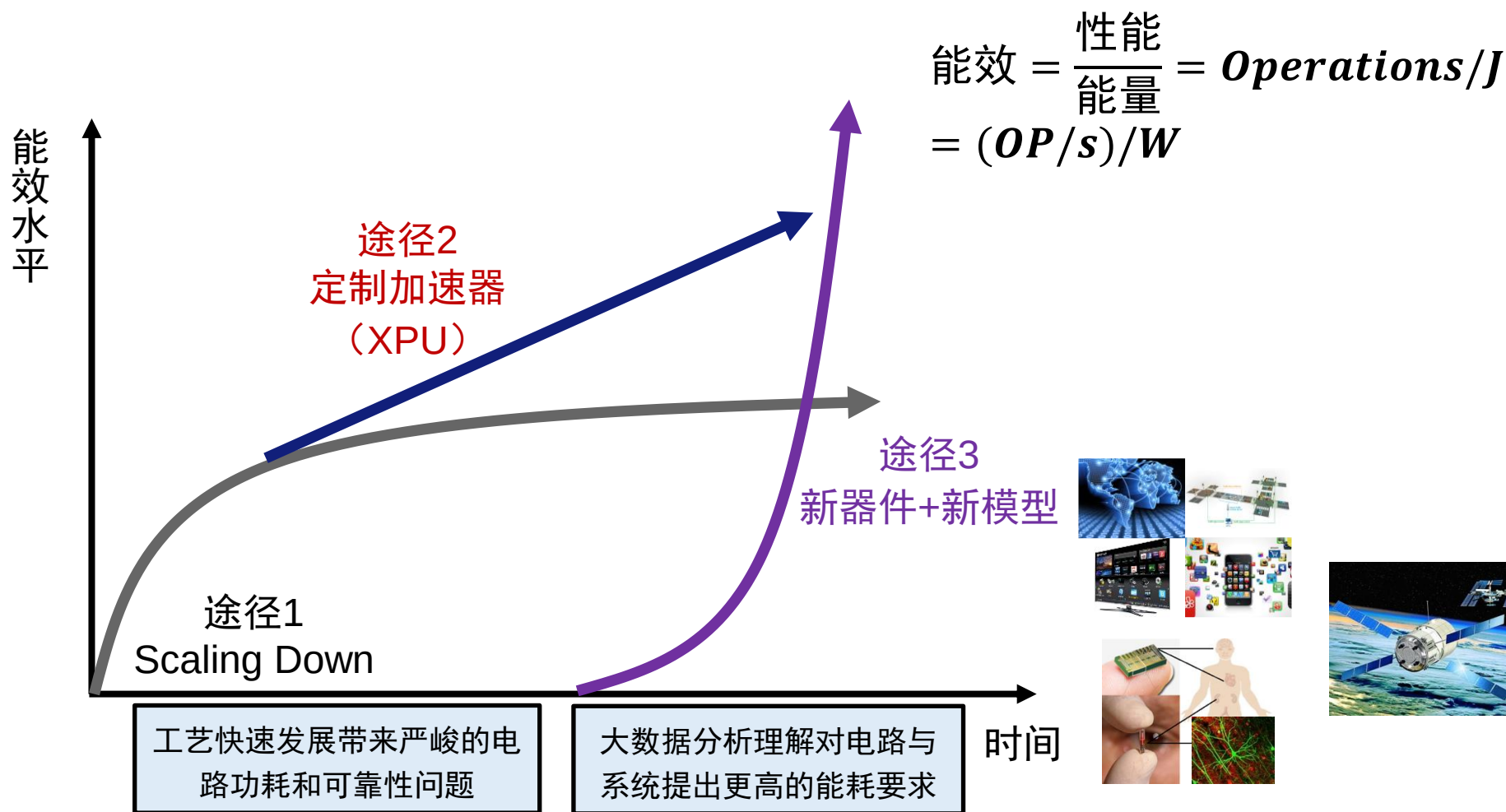


智能计算面临的矛盾

- 数据和模型的增速高于CPU和GPU性能增速
– 如何解决？



能量效率的提升途径



XPU

• 选择哪种硬件？



CPU



通用性高
能效比低



FPGA

(Field Programmable Gate Array)



功耗低；可编程
平均性能较差



GPU



性能高
功耗高；依赖大数据量



ASIC

(Application Specific Integrated Circuit)



功耗低；性能高；定制化
灵活性低

XPU

• 选择哪种硬件？

